Khazar University

Analog Control Using a Potentiometer for LED Brightness Control with Arduino UNO and Proteus Simulation

Analog Input, PWM Mapping, Circuit Scheme, Simulation, Troubleshooting, and Questions

Names

Yasin Quluzada
Ahmad Abdullayev
Aysel Ismayilova
Hikmat Mammadov
Rashada Zulfuqarli
Xumar Baghiyeva

School of Science and Engineering
Department of Computer Science
Bachelor in Computer Engineering

Khazar, May 2026

This page intentionally left blank.

Analog Control Using a Potentiometer for LED Brightness Control with Arduino UNO and Proteus Simulation

Analog Input, PWM Mapping, Circuit Scheme, Simulation, Troubleshooting, and Questions

Names
Yasin Quluzada
Ahmad Abdullayev
Aysel Ismayilova
Hikmat Mammadov
Rashada Zulfuqarli
Xumar Baghiyeva

Supervisor: Telman Askeraliyev

School of Science and Engineering
Department of Computer Science
Bachelor in Computer Engineering

Report

Khazar, May 2026

This page intentionally left blank.

Analog Control Using a Potentiometer for LED Brightness Control with Arduino UNO and Proteus Simulation

Preparation of this work was facilitated by the use of the *IPLeiria-Thesis* template.

This page intentionally left blank.

Acknowledgements

Writing Guidance

In the *Acknowledgment* section, express your gratitude to those who helped and supported your work. Start by thanking your advisors, mentors, or supervisors who provided guidance and expertise. Mention any colleagues, classmates, or team members who contributed to discussions or offered assistance. You can also acknowledge specific organisations, institutions, or funding sources that supported your research or work. Lastly, include any personal acknowledgments for family or friends who offered encouragement and moral support during the project. Keep this section sincere, concise, and professional.

This page intentionally left blank.

Abstract

This report presents the design and explanation of an analog control system in which a 10k potentiometer is used to control the brightness of a red LED through an Arduino UNO R3. The potentiometer is connected as a voltage divider between the five volt supply and ground. Its center terminal, known as the wiper, provides a variable voltage that is connected to analog input A0 of the Arduino. As the potentiometer knob is rotated, the voltage at A0 changes between approximately zero volts and five volts. The Arduino reads this analog voltage through its analog to digital conversion process and converts the reading into a pulse width modulation value. This pulse width modulation signal is produced on digital pin D9 and is applied to a red LED through a 220Ω current limiting resistor.

The purpose of the project is to demonstrate how physical movement can be converted into an electrical signal and then into a controlled output response. The report explains the project background, system concept, working principle, input and output relationship, component functions, functional block structure, signal flow, and expected system behavior. The project also prepares the foundation for later chapters involving calculation, circuit scheme, simulation, troubleshooting, and evaluation questions. The expected result is that turning the potentiometer changes the analog input voltage, which changes the pulse width modulation duty cycle, which then changes the perceived brightness of the LED.

Keywords: Analog control; potentiometer; Arduino UNO R3; analog input; analog to digital conversion; pulse width modulation; LED brightness control; voltage divider; current limiting resistor; Proteus simulation; embedded systems; circuit design; signal flow; troubleshooting.

This page intentionally left blank.

Contents

This page intentionally left blank.

List of Figures

This page intentionally left blank.

List of Tables

This page intentionally left blank.

List of Abbreviations

Ω	Ohm. (*p. xv*)
A0	Arduino analog input pin zero. (*p. xv*)
ADC	Analog to Digital Converter. (*p. xv*)
D1	Diode or LED reference label. (*p. xv*)
D9	Arduino digital pin nine. (*p. xv*)
GND	Ground reference point. (*p. xv*)
LED	Light Emitting Diode. (*p. xv*)
mA	Milliampere. (*p. xv*)
PWM	Pulse Width Modulation. (*p. xv*)
R1	Resistor reference label. (*p. xv*)
RV1	Potentiometer reference label. (*p. xv*)
UNO	Arduino UNO development board. (*p. xv*)
V	Volt. (*p. xv*)

This page intentionally left blank.

1 Description

1.1 Introduction

Analog control is an essential concept in electronics and embedded systems because many real world inputs do not naturally exist as simple on or off states. Physical movement, temperature, light, pressure, and sound often vary gradually. A microcontroller must therefore be able to read changing electrical values and convert them into useful output actions. This project demonstrates that principle through a simple but technically meaningful circuit in which a potentiometer controls the brightness of an LED.

The project uses an Arduino UNO R3 as the controller, a 10k potentiometer as the analog input device, a red LED as the visible output device, and a 220Ω resistor as the current limiting protection element. The potentiometer produces a variable voltage at its center terminal when the knob is rotated. This voltage is connected to analog input A0 of the Arduino. The Arduino reads the voltage as a numerical value and uses that value to generate a pulse width modulation signal on digital pin D9. The pulse width modulation signal changes the apparent brightness of the LED.

This chapter describes the system before detailed calculation, schematic design, simulation, and troubleshooting are discussed in later chapters. Its purpose is to give the reader a clear conceptual foundation for understanding how the project operates.

1.2 Background of Analog Control

Analog control refers to the use of continuously variable electrical signals to influence the behavior of an electronic system. Unlike digital control, which usually represents information through two states, analog control allows a signal to take many possible values within a defined range. In this project, the analog signal is the voltage at the potentiometer wiper. That voltage can vary gradually from approximately zero volts to approximately five volts.

The Arduino UNO cannot directly understand mechanical rotation. It can only measure electrical quantities at its input pins. The potentiometer solves this problem by converting rotational movement into a variable voltage. The Arduino then converts this voltage into a digital number using its analog to digital converter. This process

allows a physical action, turning a dial, to become a usable input for a programmed electronic system.

The output side uses pulse width modulation. The Arduino UNO does not produce a true continuously variable analog voltage from digital pin D9. Instead, it rapidly switches the pin between low and high states. By changing the percentage of time the signal remains high, the Arduino changes the average energy delivered to the LED. The human eye perceives this change as a change in brightness.

1.3 Project Overview

The project is an Arduino based LED brightness control system. Its main function is to allow the user to manually adjust LED brightness by rotating a potentiometer. The potentiometer acts as a control dial. The Arduino acts as the processing unit. The LED acts as the output indicator. The resistor protects the LED and the Arduino output pin from excessive current.

The circuit has two major parts. The first part is the input section, where the potentiometer is connected between five volts and ground. The center wiper terminal is connected to Arduino analog input A0. The second part is the output section, where Arduino digital pin D9 drives the LED through a 220Ω resistor. The LED cathode is returned to ground.

The system behavior is direct and observable. When the potentiometer is turned toward the ground side, the analog input voltage decreases and the LED becomes dimmer. When the potentiometer is turned toward the five volt side, the analog input voltage increases and the LED becomes brighter. This makes the project suitable for studying the relationship between physical control, analog measurement, digital processing, and output control.

1.4 Aim of the Project

The aim of this project is to design and explain an analog control circuit in which a potentiometer controls the brightness of an LED through an Arduino UNO R3. The project aims to show how mechanical rotation can be converted into a variable voltage, how that voltage can be read by a microcontroller, and how the resulting digital value can control an output through pulse width modulation.

The project also aims to prepare a complete academic explanation of the system. This includes identifying the components, explaining the input and output relationship, describing the working principle, defining the signal flow, and predicting the expected system behavior. The final goal is to produce a circuit that is conceptually clear, electrically safe, and suitable for simulation in Proteus.

1.5 Objectives of the Project

The objectives of the project are to explain the role of a potentiometer as an analog input device, to describe the Arduino UNO R3 as the controller, and to demonstrate how an LED can be controlled using pulse width modulation. The project also aims to

show the importance of correct wiring, especially the connection of the potentiometer center wiper to analog input A0.

Another objective is to understand the relationship between analog voltage and numerical readings. The Arduino analog input converts a voltage between zero volts and five volts into a value between zero and 1023. This value can then be mapped to a pulse width modulation value between zero and 255. The mathematical relationship can be expressed as:

$$N_{ADC} = \frac{V_{A0}}{V_{REF}} \times 1023$$

In this expression, N_{ADC} is the analog to digital converter reading, V_{A0} is the voltage at analog input A0, and V_{REF} is the reference voltage, normally five volts in this project.

The pulse width modulation value can be expressed as:

$$N_{PWM} = \frac{N_{ADC}}{1023} \times 255$$

These equations show that the project is not only a wiring exercise. It is also a conversion system that connects physical movement, analog voltage, digital representation, and output control.

1.6 Scope of the Project

The scope of this project is limited to controlling the brightness of a single red LED using one potentiometer and one Arduino UNO R3. The circuit uses a 10k potentiometer as the analog input, a 220Ω resistor as the current limiting resistor, and digital pin D9 as the pulse width modulation output pin.

The report focuses on conceptual explanation, component function, exact wiring logic, expected behavior, and the foundation for later calculation and simulation. The project does not include advanced motor control, wireless communication, closed loop feedback, sensor calibration, display modules, or external power electronics. These can be considered possible extensions, but they are outside the main scope of this report.

The scope is intentionally focused because the main educational purpose is to understand analog input control clearly. A narrow scope allows the report to explain every connection and every functional step with precision.

1.7 Importance of the Project

This project is important because it introduces several fundamental ideas in electronics and embedded systems through a small and understandable circuit. It shows how a variable physical action can be converted into a measurable voltage. It also shows how a microcontroller can interpret that voltage and use it to control an output.

The project is also important because it teaches correct circuit protection. The LED must not be connected directly to the Arduino output pin without a resistor. The 220Ω resistor limits current and protects both the LED and the microcontroller pin. This makes the project valuable not only for learning control, but also for learning safe circuit design.

Another important aspect is troubleshooting. A common mistake is connecting the Arduino analog input to one of the outer potentiometer pins instead of the center wiper. If this happens, the analog input receives a fixed voltage rather than a variable voltage, and the LED brightness does not change. Understanding this fault strengthens practical diagnostic ability.

1.8 System Concept

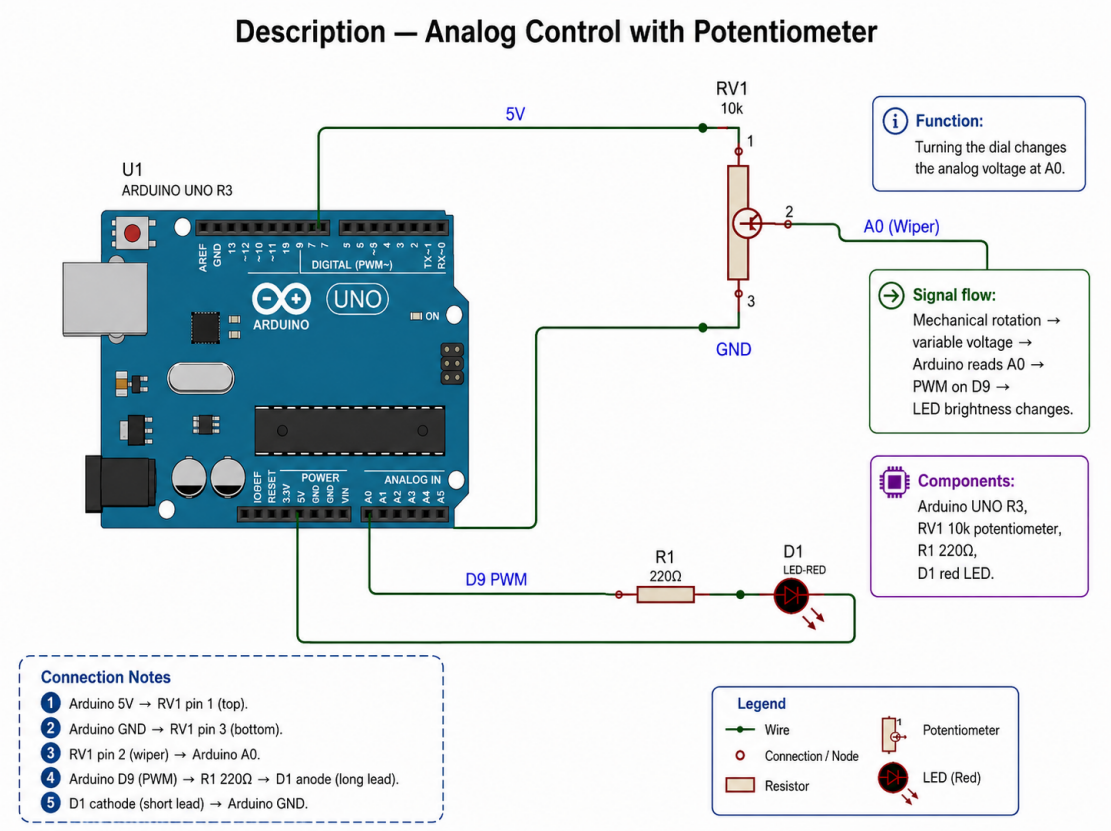

Figure 1.1: *Description - Analog Control with Potentiometer*

The system concept is based on a simple control relationship. The potentiometer is the user controlled input. The Arduino is the processing unit. The LED is the output. The user changes the potentiometer position by rotating the knob. This changes the voltage at the wiper terminal. The Arduino reads the voltage and converts it into a digital value. The Arduino then generates a pulse width modulation signal on D9. The LED receives this controlled signal through a resistor and changes brightness.

The system can be represented by the following conceptual sequence:

User rotation produces potentiometer position. Potentiometer position produces wiper voltage. Wiper voltage produces analog input reading. Analog input reading produces pulse width modulation value. Pulse width modulation value produces LED brightness.

This concept is important because it separates physical input, electrical signal, digital processing, and visible output. Each stage has a clear function and depends on the correct operation of the previous stage.

1.9 Working Principle

The working principle begins with the potentiometer. A potentiometer has three terminals. The two outer terminals are connected to the ends of a resistive track. In this project, one outer terminal is connected to five volts and the other outer terminal is connected to ground. The center terminal is the wiper. As the knob rotates, the wiper moves along the resistive track and produces a voltage between the two end values.

If the potentiometer position is represented by (p), where $p = 0$ means the wiper is at the ground side and $p = 1$ means the wiper is at the five volt side, then the voltage at A0 can be expressed as:

$$V_{A0} = p \times 5V$$

When $p = 0$, the voltage at A0 is approximately zero volts. When $p = 0.5$, the voltage at A0 is approximately 2.5 volts. When $p = 1$, the voltage at A0 is approximately five volts.

The Arduino reads this voltage using its analog input. The reading is then converted into a pulse width modulation value. A low input voltage produces a low pulse width modulation value, which makes the LED dim or off. A high input voltage produces a high pulse width modulation value, which makes the LED brighter.

1.10 Input and Output Explanation

The project has one main input and one main output. The input is the potentiometer position, represented electrically by the voltage at analog pin A0. The output is the LED brightness, controlled electrically by the pulse width modulation signal on digital pin D9.

The input and output are not directly connected. The potentiometer does not directly power the LED. Instead, the potentiometer sends information to the Arduino in the form of voltage. The Arduino processes this voltage and produces a separate output signal for the LED. This distinction is essential because it shows that the Arduino is acting as a controller rather than as a passive wire between the potentiometer and the LED.

1.10.1 Potentiometer as Input

The potentiometer is the input device of the system. Its function is to convert mechanical rotation into a variable voltage. The two outer terminals create the voltage range, while the center wiper produces the actual signal used by the Arduino.

The correct connection is essential. One outer terminal must be connected to five volts, the other outer terminal must be connected to ground, and the center terminal must be connected to A0. If A0 is connected to an outer terminal instead of the center terminal, the Arduino reads a fixed voltage and the control function fails.

The potentiometer value of 10k is suitable for this project because it provides a stable input without drawing excessive current from the five volt supply. The current through the potentiometer track can be estimated as:

$$I_{pot} = \frac{5V}{10000\Omega}$$

$$I_{pot} = 0.0005A$$

$$I_{pot} = 0.5mA$$

This small current makes the potentiometer practical for low power control input.

1.10.2 Arduino UNO as Controller

The Arduino UNO R3 is the controller of the system. Its role is to read the analog voltage from A0, convert that voltage into a numerical value, and generate a pulse width modulation signal on D9. The Arduino is therefore the decision making and signal conversion element of the project.

The analog input reading changes according to the voltage at A0. If the reference voltage is five volts, the approximate analog reading is:

$$N_{ADC} = \frac{V_{A0}}{5V} \times 1023$$

If $V_{A0} = 2.5V$, then:

$$N_{ADC} = \frac{2.5V}{5V} \times 1023$$

$$N_{ADC} = 511.5$$

The value is approximately 512.

The Arduino then maps this reading to a pulse width modulation range from zero to 255. If the analog reading is approximately 512, the pulse width modulation value is approximately 128. This corresponds to about half brightness for the LED, although the exact perceived brightness depends on the LED and human visual response.

1.10.3 LED as Output

The LED is the output device of the system. It converts electrical energy into visible light. Its brightness changes according to the pulse width modulation signal produced by digital pin D9. When the duty cycle is low, the LED appears dim. When the duty cycle is high, the LED appears bright.

The LED must be connected with correct polarity. The anode must be connected toward the resistor and Arduino D9 side. The cathode must be connected to ground. If the LED is reversed, it will not conduct properly and will not light as expected.

The LED must also be protected by a current limiting resistor. In this project, a 220Ω resistor is used. Assuming a five volt output and a red LED forward voltage of approximately two volts, the expected current is:

$$I_{LED} = \frac{5V - 2V}{220\Omega}$$

$$I_{LED} = \frac{3V}{220\Omega}$$

$$I_{LED} \approx 0.0136A$$

$$I_{LED} \approx 13.6mA$$

This value is suitable for a typical educational LED circuit and avoids excessive current.

1.11 Component Description

The component description section identifies each physical element used in the project and explains its specific function within the system. This section is necessary because the project can only be understood correctly when each component is assigned a precise role. The Arduino is not simply a power source, the potentiometer is not simply a knob, the resistor is not optional, and the LED is not merely a visual decoration. Each component participates in the control chain.

1.11.1 Arduino UNO R3

The Arduino UNO R3 is a microcontroller development board used as the main controller in this project. It receives the variable voltage from the potentiometer through analog input A0. It then converts this analog voltage into a digital reading and uses the reading to control pulse width modulation output on D9.

The board provides the five volt supply used by the potentiometer, the ground reference used by the entire circuit, the analog input used for measurement, and the digital pulse width modulation output used for LED brightness control. In this project, the Arduino therefore performs four connected functions: power reference, signal measurement, numerical processing, and output control.

1.11.2 10k Potentiometer

The 10k potentiometer is the manual control element. It contains a resistive track and a movable wiper. The two outer terminals are connected across five volts and ground. The center terminal produces a voltage that depends on the knob position.

The potentiometer works as a voltage divider. When the wiper is closer to ground, the voltage at A0 is low. When the wiper is closer to five volts, the voltage at A0 is high. This changing voltage becomes the analog input signal for the Arduino.

The 10k value is appropriate because it does not draw large current and still provides a usable voltage signal. Its function is not to directly dim the LED, but to provide the Arduino with a variable control voltage.

1.11.3 220Ω Resistor

The 220Ω resistor is the current limiting component in the LED branch. It is placed in series between Arduino D9 and the LED anode. Its function is to reduce the current flowing through the LED and the Arduino output pin.

Without this resistor, the LED could draw excessive current. This could damage the LED, stress the Arduino pin, or cause unstable circuit behavior. The resistor therefore protects the circuit and makes the output stage electrically safe.

The resistor value is justified by the relationship between supply voltage, LED forward voltage, and desired current. For a red LED with an approximate forward voltage of two volts, the 220Ω resistor gives a current close to 13.6mA at full output. This value is reasonable for a simple Arduino LED circuit.

1.11.4 Red LED

The red LED is the visible output component of the system. It emits light when current flows through it in the correct direction. Its brightness depends on the pulse width modulation signal applied from the Arduino through the resistor.

The LED has polarity. The anode must be connected toward the positive driving side, and the cathode must be connected toward ground. If the polarity is reversed, the LED will not operate correctly. In the project scheme, the D9 output connects to the resistor, the resistor connects to the LED anode, and the LED cathode connects to ground.

The red LED is suitable for this project because it provides immediate visual feedback. The user can directly observe the effect of changing the potentiometer position.

1.11.5 Jumper Wires and Ground Connection

Jumper wires provide the electrical connections between the Arduino, potentiometer, resistor, and LED. Although they are simple components, their correctness is essential. A single wrong wire can cause the circuit to fail.

The ground connection is especially important. The potentiometer ground and the LED ground must share the same Arduino ground reference. Without a common ground, the Arduino may not read the potentiometer voltage correctly, and the LED output circuit may not behave as expected.

The ground connection gives the entire circuit a common reference point. All voltage measurements in the project are meaningful only because they are measured relative to ground.

1.12 Functional Block Explanation

The system can be divided into four functional blocks: input block, processing block, output driving block, and visual output block.

The input block is the potentiometer. It receives mechanical rotation from the user and produces a variable voltage. The processing block is the Arduino UNO R3. It reads the analog voltage, converts it into a digital value, and maps that value into a pulse width modulation value. The output driving block is the D9 pin and 220Ω resistor.

This part delivers controlled current to the LED. The visual output block is the red LED, which changes brightness according to the applied pulse width modulation duty cycle.

This block structure is useful because it shows that the project is not only a set of wires. It is a complete control system with input, processing, and output stages.

1.13 Signal Flow Explanation

The signal flow begins with the user rotating the potentiometer knob. This rotation changes the position of the wiper on the resistive track. The wiper voltage changes according to its position between five volts and ground. This voltage travels through the wire connected to analog input A0.

The Arduino reads the voltage at A0 and produces an analog to digital converter value. A low voltage produces a small value, while a high voltage produces a large value. The Arduino then maps this value to a pulse width modulation value for D9. The D9 pin produces a rapidly switching signal whose duty cycle depends on the mapped value.

The signal then passes through the 220Ω resistor and reaches the LED. The resistor limits the current. The LED converts the controlled electrical energy into light. The final visible result is brightness change.

The signal path can be stated as:

Potentiometer rotation to wiper voltage. Wiper voltage to Arduino A0. Arduino A0 reading to numerical value. Numerical value to D9 pulse width modulation. D9 pulse width modulation to LED current. LED current to visible brightness.

1.14 Expected System Behavior

The expected behavior is that the LED brightness changes smoothly as the potentiometer is rotated. When the potentiometer is at the low end, the voltage at A0 should be close to zero volts. The analog reading should be close to zero, the pulse width modulation value should be close to zero, and the LED should be off or very dim.

When the potentiometer is near the middle position, the voltage at A0 should be close to 2.5 volts. The analog reading should be approximately 512, the pulse width modulation value should be approximately 128, and the LED should appear at medium brightness.

When the potentiometer is at the high end, the voltage at A0 should be close to five volts. The analog reading should be close to 1023, the pulse width modulation value should be close to 255, and the LED should appear bright.

The expected relationship can be summarized as:

$$V_{A0} \uparrow \Rightarrow N_{ADC} \uparrow \Rightarrow N_{PWM} \uparrow$$
$$\Rightarrow Brightness \uparrow$$

This means that increasing the potentiometer output voltage increases the analog reading, increases the pulse width modulation value, and increases the LED brightness. If the LED brightness does not change when the potentiometer is rotated, the

most likely fault is an incorrect connection at the potentiometer wiper, an incorrect output connection, an LED polarity error, a missing ground, or an issue in the Arduino program.

1.15 Chapter Summary

This chapter described the analog control project in formal technical terms. The system uses a 10k potentiometer as a variable analog input, an Arduino UNO R3 as the controller, a 220Ω resistor as the current limiting element, and a red LED as the visual output. The potentiometer converts mechanical rotation into a variable voltage at its center wiper. The Arduino reads this voltage through analog input A0, converts it into a digital value, maps it into a pulse width modulation output value, and applies the resulting signal through D9 to the LED circuit.

The chapter also explained the importance of correct wiring, especially the connection of the potentiometer center terminal to A0 and the use of a common ground. The expected system behavior is that rotating the potentiometer changes the A0 voltage, which changes the pulse width modulation duty cycle, which changes LED brightness. This description provides the conceptual foundation for the next chapters, where the numerical calculations, circuit scheme, simulation, troubleshooting process, and academic questions are developed in greater detail.

2 Calculation

2.1 Introduction to Electrical Calculations

This chapter presents the electrical calculations required to justify the operation of the analog control circuit. The purpose of the calculation chapter is to prove that the selected components, signal ranges, voltage levels, current limits, and control values are electrically valid for the intended Arduino based LED dimming system.

The circuit is based on a 10k potentiometer connected as a voltage divider, an Arduino UNO R3 used as the controller, and a red LED driven from a pulse width modulation output through a 220Ω current limiting resistor. The calculation process begins with the potentiometer output voltage, continues with the Arduino analog to digital conversion, then proceeds to pulse width modulation mapping, and finally verifies the LED resistor value and expected current.

The calculation chapter is necessary because a circuit diagram alone does not prove electrical correctness.
A formal design must show that the analog input voltage remains within the safe input range of the Arduino, that the analog reading is correctly interpreted, that the pulse width modulation value is correctly derived, and that the LED current remains within a safe operating range.

In this project, the main electrical sequence is as follows:

Potentiometer position produces a variable voltage at A0. Arduino reads the A0 voltage as a digital value from 0 to 1023. The digital value is converted into a pulse width modulation value from 0 to 255. The pulse width modulation signal controls the average brightness of the LED. The resistor limits the LED current to a safe value.

Therefore, the calculations in this chapter provide the numerical foundation of the whole project.

2.2 Potentiometer as a Voltage Divider

A potentiometer is a three terminal variable resistor. In this project, it is used as a voltage divider rather than as a simple variable resistor. One outer terminal of the potentiometer is connected to the 5V supply of the Arduino. The other outer terminal

is connected to ground. The center terminal, known as the wiper, is connected to the Arduino analog input pin A0.

The voltage at the wiper depends on the position of the rotating shaft. When the wiper is closer to the ground side, the voltage at A0 is lower. When the wiper is closer to the 5V side, the voltage at A0 is higher. This allows mechanical rotation to be converted into an electrical voltage.

The total resistance of the potentiometer is 10kΩ. Since the potentiometer is connected across 5V and ground, the total current through the potentiometer is:

$$I_{pot} = \frac{V}{R}$$

$$I_{pot} = \frac{5V}{10000\Omega}$$

$$I_{pot} = 0.0005A$$

$$I_{pot} = 0.5mA$$

This value is small enough for an Arduino based educational circuit. The potentiometer therefore provides a stable variable voltage while drawing only a small amount of current from the 5V supply.

The important point is that Arduino A0 does not directly measure the resistance of the potentiometer. It measures the voltage produced at the wiper. The resistance change is only the internal mechanism that creates a changing voltage.

2.3 Potentiometer Output Voltage Calculation

The potentiometer output voltage is determined by the position of the wiper between the 5V terminal and the ground terminal. If the position of the wiper is represented by (p), where $p = 0$ means the wiper is at the ground side and $p = 1$ means the wiper is at the 5V side, then the output voltage at A0 is:

$$V_{A0} = 5V \times p$$

This equation shows that the analog input voltage is proportional to the mechanical position of the potentiometer.

If the potentiometer is rotated from minimum to maximum, the A0 voltage changes continuously from approximately 0V to approximately 5V. This is the required behavior for analog control.

2.3.1 Minimum Position Calculation

At the minimum position, the wiper is assumed to be at the ground side of the potentiometer. Therefore:

$$p = 0$$

$$V_{A0} = 5V \times 0$$

$$V_{A0} = 0V$$

At this position, the Arduino analog input receives approximately 0V. The expected analog reading is therefore the minimum possible value, and the LED should be off or almost off.

2.3.2 Middle Position Calculation

At the middle position, the wiper is assumed to be halfway between 5V and ground. Therefore:

$$p = 0.5$$

$$V_{A0} = 5V \times 0.5$$

$$V_{A0} = 2.5V$$

At this position, the Arduino analog input receives approximately half of the reference voltage. The expected analog reading is therefore approximately half of the maximum ADC value, and the LED should appear at approximately medium brightness.

2.3.3 Maximum Position Calculation

At the maximum position, the wiper is assumed to be at the 5V side of the potentiometer. Therefore:

$$p = 1$$

$$V_{A0} = 5V \times 1$$

$$V_{A0} = 5V$$

At this position, the Arduino analog input receives approximately 5V. The expected analog reading is therefore the maximum possible value, and the LED should appear fully bright.

2.4 Arduino Analog Input Range

The Arduino UNO analog input pin A0 is designed to read voltage values within the range determined by the analog reference voltage. In the standard Arduino UNO con-

figuration, the reference voltage is 5V. Therefore, the valid analog input range is from 0V to 5V.

The input voltage must not be lower than ground and must not exceed the reference voltage under normal circuit operation. In this project, the potentiometer output is naturally limited between ground and 5V because its two outer terminals are connected to those two boundaries. This makes the potentiometer voltage safe and suitable for direct connection to A0.

The valid input range is therefore:

$$0V \leq V_{A0} \leq 5V$$

This condition confirms that the potentiometer output is compatible with the Arduino analog input.

2.5 ADC Resolution Explanation

The Arduino UNO uses an analog to digital converter to transform the analog voltage at A0 into a numerical value. The analog to digital converter has a 10 bit resolution. A 10 bit converter provides:

$$2^{10} = 1024$$

possible numerical levels.

Since counting begins from zero, the digital output range is:

$$0 \text{ to } 1023$$

This means that 0V corresponds approximately to an ADC value of 0, while 5V corresponds approximately to an ADC value of 1023. Each step represents a small voltage interval.

The voltage represented by one ADC step is:

$$V_{step} = \frac{5V}{1023}$$

$$V_{step} \approx 0.004887V$$

$$V_{step} \approx 4.89mV$$

Therefore, each increase of one ADC count represents approximately 4.89mV of input voltage. This resolution is sufficient for smooth LED brightness control in this project.

2.6 ADC Value Calculation

The ADC value is calculated by comparing the input voltage at A0 with the reference voltage. Since the reference voltage is 5V and the ADC range is from 0 to 1023, the

ADC value is:

$$ADC = \frac{V_{A0}}{5V} \times 1023$$

This formula converts the analog voltage into the digital reading used by the Arduino program.

2.6.1 ADC Value at 0V

When the potentiometer output is 0V:

$$ADC = \frac{0V}{5V} \times 1023$$

$$ADC = 0$$

Therefore, an input voltage of 0V produces an ADC value of 0.

2.6.2 ADC Value at 1.25V

When the potentiometer output is 1.25V:

$$ADC = \frac{1.25V}{5V} \times 1023$$

$$ADC = 0.25 \times 1023$$

$$ADC = 255.75$$

The nearest practical integer value is:

$$ADC \approx 256$$

Therefore, an input voltage of 1.25V produces an ADC value of approximately 256.

2.6.3 ADC Value at 2.5V

When the potentiometer output is 2.5V:

$$ADC = \frac{2.5V}{5V} \times 1023$$

$$ADC = 0.5 \times 1023$$

$$ADC = 511.5$$

The nearest practical integer value is:

$$ADC \approx 512$$

Therefore, an input voltage of 2.5V produces an ADC value of approximately 512.

2.6.4 ADC Value at 3.75V

When the potentiometer output is 3.75V:

$$ADC = \frac{3.75V}{5V} \times 1023$$

$$ADC = 0.75 \times 1023$$

$$ADC = 767.25$$

The nearest practical integer value is:

$$ADC \approx 767$$

Therefore, an input voltage of 3.75V produces an ADC value of approximately 767.

2.6.5 ADC Value at 5V

When the potentiometer output is 5V:

$$ADC = \frac{5V}{5V} \times 1023$$

$$ADC = 1023$$

Therefore, an input voltage of 5V produces the maximum ADC value of 1023.

2.7 PWM Output Range

The Arduino UNO does not produce a true continuously variable analog voltage from a normal digital pin. Instead, it produces a pulse width modulation signal. Pulse width modulation controls the average power delivered to the LED by switching the output rapidly between LOW and HIGH.

In Arduino programming, the analogWrite function uses a value from 0 to 255 for standard PWM output. Therefore, the PWM output range is:

$$0 \leq PWM \leq 255$$

A PWM value of 0 means the output remains LOW, so the LED is off. A PWM value of 255 means the output is fully HIGH for the complete cycle, so the LED is at maximum brightness. Values between 0 and 255 create intermediate brightness levels.

The brightness is controlled by changing the duty cycle, not by creating a true analog voltage.

2.8 ADC to PWM Mapping

The analog input value has a range from 0 to 1023, while the PWM output value has a range from 0 to 255. Therefore, the ADC value must be converted into the PWM range.

A simple conversion is:

$$PWM = \frac{ADC}{4}$$

This works because:

$$\frac{1023}{4} = 255.75$$

Since the maximum PWM value is 255, the result is limited to the 8 bit PWM range.

A more formal mapping equation is:

$$PWM = \frac{ADC \times 255}{1023}$$

This equation directly maps the full ADC input range to the full PWM output range.

Examples are as follows:

$$ADC = 0 \Rightarrow PWM = 0$$

$$ADC = 256 \Rightarrow PWM \approx 64$$

$$ADC = 512 \Rightarrow PWM \approx 128$$

$$ADC = 767 \Rightarrow PWM \approx 191$$

$$ADC = 1023 \Rightarrow PWM = 255$$

This mapping ensures that a low potentiometer voltage produces low LED brightness, a middle voltage produces medium brightness, and a high voltage produces high brightness.

2.9 PWM Duty Cycle Explanation

Pulse width modulation is based on the duty cycle of a digital signal. The duty cycle is the percentage of time that the signal remains HIGH during one complete cycle.

The duty cycle can be calculated from the PWM value as follows:

$$Duty\ Cycle = \frac{PWM}{255} \times 100$$

If the PWM value is 0:

$$Duty\ Cycle = \frac{0}{255} \times 100 = 0\%$$

If the PWM value is 128:

$$Duty\ Cycle = \frac{128}{255} \times 100$$

$$Duty\ Cycle \approx 50.2\%$$

If the PWM value is 255:

$$Duty\ Cycle = \frac{255}{255} \times 100 = 100\%$$

The LED appears dimmer or brighter because the human eye perceives the average light output over time. At a low duty cycle, the LED is on for a smaller part of each cycle, so the brightness appears low. At a high duty cycle, the LED is on for most of each cycle, so the brightness appears high.

Thus, PWM controls perceived LED brightness by controlling average delivered power.

2.10 LED Forward Voltage Assumption

A red LED requires a forward voltage before it conducts current and emits light. For this project, the forward voltage of the red LED is assumed to be approximately:

$$V_{LED} = 2V$$

This value is a standard practical assumption for a typical red LED in an educational circuit. The exact value may vary slightly depending on the LED model, manufacturing tolerance, and operating current. However, 2V is a suitable design assumption for selecting a safe current limiting resistor.

The Arduino output voltage when the digital pin is HIGH is assumed to be approximately:

$$V_{OUT} = 5V$$

Therefore, the voltage remaining across the resistor is:

$$V_R = V_{OUT} - V_{LED}$$

$$V_R = 5V - 2V$$

$$V_R = 3V$$

This remaining voltage is the voltage that determines the LED current through the resistor.

2.11 LED Current Limiting Resistor Calculation

The current limiting resistor is required to prevent excessive current through the LED and the Arduino output pin. Without this resistor, the LED could draw too much current, which may damage the LED or the microcontroller output pin.

The resistor value is calculated using Ohm's law:

$$R = \frac{V}{I}$$

For the LED branch, the voltage across the resistor is:

$$V_R = 3V$$

A suitable target current for an educational red LED circuit is approximately:

$$I = 0.015A$$

This equals:

$$I = 15mA$$

Therefore, the required resistor value is:

$$R = \frac{3V}{0.015A}$$

$$R = 200\Omega$$

Since 200Ω is not always the preferred common value in basic component sets, a 220Ω resistor is selected.

2.12 Justification for Selecting 220Ω Resistor

The selected resistor value is:

$$R = 220\Omega$$

This value is slightly higher than the calculated 200Ω value. A higher resistance reduces the current slightly, which improves safety for both the LED and the Arduino output pin.

The use of 220Ω is justified for four reasons.

First, it limits the LED current to a safe value. Second, it protects the Arduino output pin from excessive current. Third, it provides enough current for visible LED brightness. Fourth, it is a standard resistor value commonly available in electronics kits and simulation libraries.

The 220Ω resistor therefore represents a safe and practical design choice.

2.13 Expected LED Current Calculation

Using the selected resistor value of 220Ω, the expected LED current at full output can be calculated as follows:

$$I = \frac{V_R}{R}$$

$$I = \frac{3V}{220\Omega}$$

$$I = 0.013636A$$

Converting this value to milliamperes:

$$I \approx 13.6mA$$

Therefore, the expected LED current at full brightness is approximately 13.6mA.

This current is appropriate for a typical red LED and is suitable for an Arduino controlled educational circuit. It is high enough to produce visible brightness, but low enough to remain within a safe practical range.

2.14 Calculation Table

The following table summarizes the relationship between potentiometer position, A0 voltage, ADC value, PWM value, duty cycle, and expected LED behavior.

Table 2.1: *Calculation table*

Potentiometer Position	A0 Voltage	ADC Value	PWM Value	Duty Cycle	Expected LED Behavior
0 percent	0V	0	0	0 percent	Off
25 percent	1.25V	256	64	About 25 percent	Low brightness
50 percent	2.5V	512	128	About 50 percent	Medium brightness
75 percent	3.75V	767	191	About 75 percent	High brightness
100 percent	5V	1023	255	100 percent	Full brightness

This table confirms the expected proportional relationship between potentiometer rotation and LED brightness. As the potentiometer position increases, the A0 voltage increases. As the A0 voltage increases, the ADC reading increases. As the ADC reading increases, the PWM value increases. As the PWM value increases, the LED brightness increases.

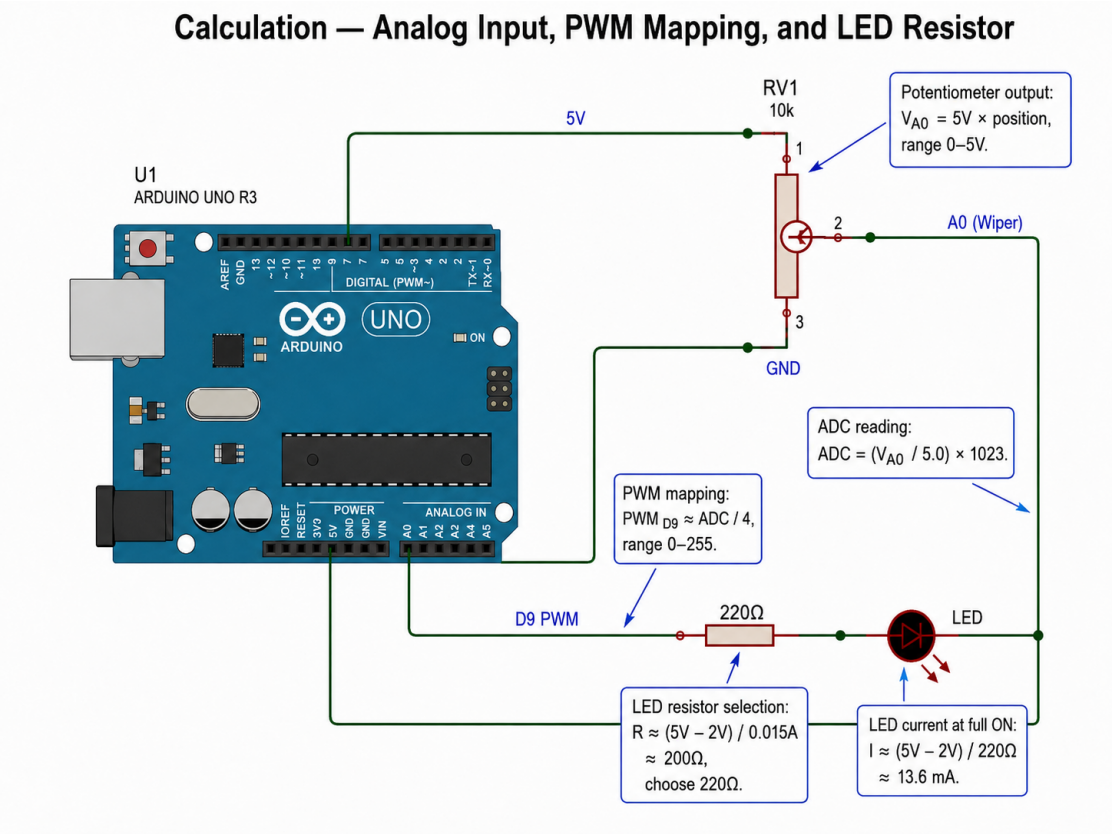

Figure 2.1: *Calculation - Analog Input, PWM Mapping, and LED Resistor*

2.15 Calculation Verification

The calculations are verified by checking whether each electrical stage remains inside its required operating boundary.

The potentiometer output voltage is verified because it remains within the following range:

$$0V \leq V_{A0} \leq 5V$$

This is the valid analog input range for the Arduino UNO when the analog reference is 5V.

The ADC values are verified because they remain within the 10 bit range:

$$0 \leq ADC \leq 1023$$

The PWM values are verified because they remain within the 8 bit output range:

$$0 \leq PWM \leq 255$$

The LED resistor is verified because it limits the expected current to approximately:

$$I \approx 13.6mA$$

This current value is suitable for the LED and acceptable for an Arduino controlled

educational output circuit.

The full calculation chain is therefore consistent:

$$Position \rightarrow V_{A0} \rightarrow ADC \rightarrow PWM \rightarrow LED\ Brightness$$

This proves that the system is mathematically coherent and electrically reasonable.

2.16 Chapter Summary

This chapter established the mathematical foundation of the analog control circuit. The potentiometer was analyzed as a voltage divider that produces a variable voltage between 0V and 5V. This voltage is applied to Arduino analog input A0, where it is converted into a 10 bit ADC value from 0 to 1023.

The ADC value is then mapped into an 8 bit PWM value from 0 to 255. This PWM value controls the duty cycle of digital pin D9, which determines the perceived brightness of the LED. The LED branch was also analyzed using Ohm's law. With a 5V output, an assumed red LED forward voltage of 2V, and a selected resistor of 220Ω, the expected LED current is approximately 13.6mA.

The calculations confirm that the circuit is safe, logical, and suitable for simulation and practical demonstration. The potentiometer provides a valid analog signal, the Arduino reads and maps that signal correctly, and the LED branch is protected by an appropriate resistor. Therefore, the calculation chapter supports the technical validity of the complete analog control project.

3 Scheme

3.1 Introduction to the Circuit Scheme

This chapter presents the complete circuit scheme for the analog control system using an Arduino UNO R3, a 10 kΩ potentiometer, a 220 Ω resistor, and a red light emitting diode. The scheme defines the exact electrical relationships between the input device, the controller, and the output device. In this project, the potentiometer provides a variable analog voltage to the Arduino analog input pin A0, while the Arduino produces a pulse width modulation signal from digital pin D9 to control the brightness of the LED.

The circuit scheme is an essential part of the project because it converts the theoretical concept into a precise electrical design. Without an exact scheme, the project would remain only a conceptual explanation. The scheme establishes which component is connected to which pin, how the voltage is supplied, how the ground reference is shared, and how the output current is limited before reaching the LED.

The main function of the scheme is to show that the potentiometer is not connected as a random variable resistor. It is connected as a voltage divider between 5 V and ground. The center terminal of the potentiometer, also known as the wiper terminal, produces the variable voltage that is read by the Arduino at A0. The output side of the scheme shows that the LED is not connected directly to the Arduino pin. Instead, the LED is connected through a 220 Ω resistor to limit current and protect both the LED and the Arduino output pin.

Therefore, this chapter provides the physical and electrical foundation for the simulation and troubleshooting chapters that follow. Every simulation result and every troubleshooting case depends on the correctness of the scheme described in this chapter.

3.2 Purpose of the Scheme

The purpose of the circuit scheme is to provide a clear, accurate, and reproducible representation of the complete system. The scheme must allow another student, instructor, or reviewer to reconstruct the circuit without uncertainty. Every pin, terminal,

component value, and connection path must be explicitly defined.

The scheme has five main purposes. First, it identifies all components used in the circuit. Second, it defines the exact wiring between the Arduino UNO R3, the potentiometer, the resistor, and the LED. Third, it establishes the electrical logic of the input stage and output stage. Fourth, it prevents common wiring errors, especially the incorrect connection of the analog input pin to an outer potentiometer terminal instead of the wiper terminal. Fifth, it prepares the circuit for accurate Proteus simulation.

The scheme also proves that the circuit is electrically coherent. The input side uses a 5 V reference and ground reference to create a variable voltage at A0. The output side uses pin D9 to provide a pulse width modulation signal through a current limiting resistor to the LED. The ground connection is common to all parts of the circuit, which ensures that the analog input voltage and output signal have the same electrical reference.

The scheme therefore functions as the central bridge between the design explanation and the practical simulation. It is not only a drawing. It is the formal electrical map of the project.

3.3 Required Proteus Components

The Proteus simulation requires the following components to construct the circuit correctly. The first component is the Arduino UNO R3, which acts as the central controller. The second component is a 10 kΩ potentiometer, which acts as the analog input device. The third component is a 220 Ω resistor, which limits the LED current. The fourth component is a red LED, which acts as the visual output device. The fifth required element is the ground terminal, which provides the common electrical reference for the whole circuit.

The Arduino UNO R3 must be selected because the project depends on its analog input pin A0 and pulse width modulation output pin D9. The potentiometer must have three terminals, since the circuit requires two fixed terminals for 5 V and ground and one moving wiper terminal for the variable analog voltage. The resistor must be placed in series with the LED to restrict current flow. The LED must be correctly oriented so that its anode receives the controlled output through the resistor and its cathode returns to ground.

Optional Proteus instruments may also be included for verification. A DC voltmeter may be connected between A0 and ground to observe the potentiometer output voltage. An oscilloscope may be connected to D9 and ground to observe the pulse width modulation waveform. These instruments are not required for the basic circuit to operate, but they strengthen the academic quality of the simulation because they provide measurable evidence of circuit behavior.

3.4 Proteus Component Selection

Proteus component selection must be performed carefully because incorrect component choice can produce inaccurate simulation behavior. The components must represent the real electrical function of the physical parts used in the project. The selected

Arduino board must support analog input and pulse width modulation output. The selected potentiometer must behave as a three terminal variable resistor. The selected LED must have a realistic forward voltage model. The selected resistor must have the correct resistance value. The selected ground terminal must connect all ground points to the same electrical reference.

The component selection process must also consider the educational purpose of the project. The circuit is intended to demonstrate analog control, voltage division, analog to digital conversion, pulse width modulation, and output current limitation. Therefore, each component must be selected to make these principles visible and verifiable inside the schematic.

3.4.1 Arduino UNO R3 Selection

The Arduino UNO R3 is selected as the controller because it provides the required analog input and digital pulse width modulation output. In this circuit, analog input pin A0 reads the voltage from the potentiometer wiper terminal. Digital pin D9 produces the pulse width modulation signal that controls LED brightness.

The Arduino UNO R3 operates with a 5 V logic system in this project. This makes it suitable for the potentiometer voltage divider, since the potentiometer output voltage can vary from approximately 0 V to approximately 5 V. The analog input converts this voltage into a numerical value from 0 to 1023. The program then maps that value into a pulse width modulation output range from 0 to 255.

The Arduino UNO R3 is also appropriate because pin D9 is a pulse width modulation capable pin. This is important because LED dimming is not achieved by changing a true analog output voltage from the Arduino. Instead, it is achieved by changing the duty cycle of the pulse width modulation signal. A lower duty cycle produces lower perceived brightness, while a higher duty cycle produces higher perceived brightness.

3.4.2 Potentiometer Selection

A 10 kΩ potentiometer is selected as the analog input component. The potentiometer is used as a voltage divider rather than as a simple two terminal variable resistor. In this configuration, one outer terminal is connected to 5 V, the other outer terminal is connected to ground, and the center terminal is connected to Arduino analog input A0.

The 10 kΩ value is appropriate because it provides stable analog voltage control without drawing excessive current from the Arduino 5 V supply. The current through the potentiometer can be estimated using Ohm's law.

The supply voltage is 5 V. The potentiometer resistance is 10 kΩ. The current through the potentiometer is calculated as follows.

$$I = V \text{divided by} R$$

$$I = 5V \text{divided by} 10,000\Omega$$

$$I = 0.0005A$$

$$I = 0.5mA$$

This current is small enough for safe educational use and large enough to produce a stable voltage at the analog input. The potentiometer therefore provides a controlled and efficient analog input signal.

The center terminal of the potentiometer is the most important connection in the input stage. It produces the variable voltage. If A0 is connected to an outer terminal instead of the center terminal, the Arduino receives a fixed voltage rather than a variable voltage. In that case, rotating the potentiometer does not change the analog reading.

3.4.3 Resistor Selection

A 220 Ω resistor is selected as the current limiting resistor for the LED. The resistor is necessary because an LED must not be connected directly to an Arduino output pin. Without a resistor, the current through the LED could become excessive, which may damage the LED or overload the Arduino pin.

The resistor value can be justified using the expected voltage across the resistor. The Arduino output voltage is approximately 5 V when the output is high. A typical red LED has an approximate forward voltage of 2 V. Therefore, the voltage across the resistor is approximately 3 V.

$$Vresistor = 5V\text{minus}2V$$

$$Vresistor = 3V$$

Using a 220 Ω resistor, the LED current is calculated as follows.

$$I = V\text{divided by}R$$

$$I = 3V\text{divided by}220\Omega$$

$$I = 0.0136A$$

$$I = 13.6mA$$

This current is suitable for a basic LED brightness control project. It is high enough to produce visible illumination and low enough to protect the circuit under normal educational conditions.

3.4.4 LED Selection

A red LED is selected as the output indicator because its brightness can be easily observed during simulation. The LED converts electrical energy into visible light. In this project, its brightness depends on the pulse width modulation signal produced by Arduino pin D9.

The LED must be connected with correct polarity. The anode must be connected to the resistor side that receives the signal from D9. The cathode must be connected to ground. If the LED is reversed, it will not conduct current in the intended direction, and it will not illuminate.

The LED is not the control device in this circuit. It is the output indicator. The actual control is performed by the Arduino after reading the potentiometer voltage. The LED simply displays the result of that control process through visible brightness variation.

3.4.5 Ground Terminal Selection

The ground terminal is selected to provide a common reference point for the whole circuit. The Arduino ground, potentiometer ground, and LED cathode ground must all be electrically connected. A circuit with separate or floating ground references cannot be expected to produce reliable analog readings or correct output behavior.

The ground connection is especially important for the analog input. The Arduino measures the voltage at A0 relative to its own ground. If the potentiometer ground is not connected to Arduino ground, the voltage at A0 may become undefined or unstable. The same principle applies to the LED output. The LED current requires a complete return path back to ground.

Therefore, the ground terminal is not a minor schematic detail. It is the electrical reference that allows the input voltage, output signal, and current path to function correctly.

3.5 Complete Circuit Connection Table

The complete circuit connection table defines the exact wiring of the project. It removes ambiguity by specifying the source point, destination point, signal meaning, and purpose of every connection.

This table must be followed exactly. The most critical connection is the third connection, where potentiometer pin 2 must connect to Arduino A0. This is the only connection that provides the variable analog signal required for brightness control.

3.6 Potentiometer Wiring

The potentiometer wiring forms the input stage of the circuit. Its purpose is to convert mechanical rotation into a variable voltage. The potentiometer must be connected as a voltage divider with three active terminals. One outer terminal is connected to 5 V. The other outer terminal is connected to ground. The center terminal is connected to A0.

Table 3.1: *Complete circuit connection table*

Connection Number	Source Point	Destination Point	Signal or Function	Purpose
1	Arduino 5 V	Potentiometer pin 1	Positive voltage reference	Supplies the upper voltage boundary of the voltage divider
2	Arduino ground	Potentiometer pin 3	Ground reference	Supplies the lower voltage boundary of the voltage divider
3	Potentiometer pin 2	Arduino A0	Variable analog voltage	Sends the wiper voltage to the Arduino analog input
4	Arduino D9	220 Ω resistor	Pulse width modulation output	Sends the brightness control signal to the LED branch
5	220 Ω resistor	LED anode	Current limited output signal	Allows controlled current to enter the LED
6	LED cathode	Arduino ground	Current return path	Completes the LED circuit
7	Potentiometer ground node	Arduino ground node	Common reference	Ensures stable analog voltage measurement

When the potentiometer is rotated, the internal position of the wiper changes. This changes the ratio between the resistance above the wiper and the resistance below the wiper. As a result, the voltage at the center terminal changes between approximately 0 V and approximately 5 V.

If the potentiometer position is represented by p, where p ranges from 0 to 1, then the wiper voltage can be expressed as follows.

$$VA0 = 5V \text{multiplied by} p$$

When p equals 0, the wiper voltage is approximately 0 V. When p equals 0.5, the wiper voltage is approximately 2.5 V. When p equals 1, the wiper voltage is approximately 5 V.

This variable voltage is the input signal that allows the Arduino to determine the desired LED brightness.

3.6.1 Potentiometer Pin 1 Connection

Potentiometer pin 1 is connected to the Arduino 5 V pin. This connection provides the upper voltage reference of the voltage divider. The maximum possible voltage at the wiper terminal is therefore approximately 5 V.

This connection must be stable and direct. If pin 1 is not connected to 5 V, the potentiometer cannot provide the full input voltage range. If pin 1 is accidentally connected to ground while pin 3 is connected to 5 V, the circuit may still work, but the direction of control will be reversed. In that case, turning the knob in one direction may decrease brightness instead of increasing it.

The connection of pin 1 to 5 V does not mean that A0 always receives 5 V. A0 receives the voltage from the wiper terminal, which depends on the rotational position of the potentiometer.

3.6.2 Potentiometer Pin 2 or Wiper Connection

Potentiometer pin 2 is the wiper terminal. It must be connected to Arduino analog input A0. This is the most important connection in the entire input stage because it carries the variable analog voltage.

The wiper terminal moves electrically between the 5 V side and the ground side as the knob is rotated. Because of this movement, the voltage at pin 2 changes continuously within the allowed range. The Arduino reads this voltage and converts it into a digital value.

The analog reading can be expressed as follows.

ADC value = V A0 divided by 5 V multiplied by 1023

If the wiper voltage is 0 V, the ADC value is approximately 0. If the wiper voltage is 2.5 V, the ADC value is approximately 512. If the wiper voltage is 5 V, the ADC value is approximately 1023.

If pin 2 is left unconnected, A0 may float and produce unstable values. If A0 is connected to pin 1 or pin 3 instead of pin 2, the analog reading will remain fixed near 1023 or near 0. Therefore, pin 2 must be connected to A0 for correct operation.

3.6.3 Potentiometer Pin 3 Connection

Potentiometer pin 3 is connected to Arduino ground. This connection provides the lower voltage reference of the voltage divider. The minimum possible voltage at the wiper terminal is therefore approximately 0 V.

The ground connection must be shared with the Arduino ground. This is necessary because the Arduino analog input measures voltage relative to its own ground reference. If the potentiometer ground is not connected to Arduino ground, the voltage at A0 may not have a defined reference, and the analog reading may become unstable.

When the potentiometer is rotated toward the grounded side, the wiper voltage decreases. This causes the ADC reading to decrease, which reduces the pulse width modulation value and decreases LED brightness.

3.7 Arduino Analog Input Wiring

Arduino analog input A0 is connected to the potentiometer wiper terminal. A0 is the measurement point for the variable voltage generated by the potentiometer. The analog input does not measure resistance directly. It measures voltage relative to Arduino ground.

The analog input voltage must remain within the safe input range of the Arduino. In this project, the voltage range is approximately 0 V to 5 V because the potentiometer is connected between 5 V and ground. This makes the signal suitable for direct connection to A0.

The Arduino converts the analog voltage into a digital number. With a 10 bit analog to digital converter, the input range is divided into 1024 discrete levels, from 0 to 1023. The approximate voltage step per ADC count is calculated as follows.

Voltage step = 5 V divided by 1023

Voltage step = 0.00489 V

Voltage step is approximately 4.89 mV

This means that each increase of one ADC count represents approximately 4.89 mV of input voltage change. This resolution is sufficient for smooth LED brightness control in this project.

3.8 LED Output Wiring

The LED output wiring forms the output stage of the circuit. It connects Arduino digital pin D9 to the LED through a 220 Ω resistor. The resistor and LED are connected in series, which means the same current flows through both components when the output signal is active.

The Arduino does not vary LED brightness by producing a continuously variable output voltage. Instead, it uses pulse width modulation. A pulse width modulation signal switches rapidly between high and low states. The duty cycle determines the average power delivered to the LED. A low duty cycle produces dim light. A high duty cycle produces bright light.

The output wiring must therefore preserve three conditions. First, D9 must connect to the resistor. Second, the resistor must connect to the LED anode. Third, the

LED cathode must connect to ground. These three conditions create a protected and complete current path.

3.8.1 Arduino D9 to Resistor

Arduino digital pin D9 is connected to one side of the 220 Ω resistor. D9 is selected because it is capable of producing pulse width modulation output. The resistor must be placed immediately in series with the LED branch so that current is limited whenever the LED conducts.

The pulse width modulation value is determined by the mapped analog reading. The mapping relationship can be expressed as follows.

PWM value = ADC value multiplied by 255 divided by 1023

For practical Arduino code, this is often represented by mapping the analog range from 0 to 1023 into the output range from 0 to 255.

When the ADC value is 0, the PWM value is 0. When the ADC value is approximately 512, the PWM value is approximately 128. When the ADC value is 1023, the PWM value is 255.

This connection therefore carries the processed control signal from the Arduino to the LED output stage.

3.8.2 Resistor to LED Anode

The second side of the 220 Ω resistor is connected to the anode of the LED. The anode is the positive side of the LED. This connection ensures that current entering the LED has already passed through the current limiting resistor.

The resistor must not be bypassed. If the resistor is omitted or connected incorrectly, the LED may draw excessive current. The correct series path is D9, then resistor, then LED anode, then LED cathode, then ground.

The expected LED current at full output can be calculated as follows.

$$ILED = 5V \text{minus} 2V \text{divided by} 220\Omega$$

$$ILED = 3V \text{divided by} 220\Omega$$

I LED is approximately 0.0136 A

I LED is approximately 13.6 mA

This value is suitable for visible LED operation and protects the circuit under normal educational conditions.

3.8.3 LED Cathode to Ground

The LED cathode is connected to Arduino ground. This connection completes the output current path. Current flows from D9 through the resistor, enters the LED through the anode, exits through the cathode, and returns to ground.

If the cathode is not connected to ground, the LED circuit is open and no current can flow. If the LED is reversed, the LED will block current in the intended direction and

will not illuminate. Therefore, correct polarity and correct ground return are required for operation.

The ground return of the LED must connect to the same ground reference used by the Arduino and the potentiometer. This creates one unified circuit reference.

3.9 Common Ground Requirement

A common ground is required for the entire circuit. The Arduino, potentiometer, and LED output branch must all share the same ground reference. Without a common ground, the Arduino cannot correctly interpret the potentiometer voltage, and the LED branch may not have a valid return path.

The common ground performs two functions. In the input stage, it defines the zero volt reference for the potentiometer voltage divider. In the output stage, it completes the current path for the LED. These two functions are different, but they depend on the same electrical reference.

The common ground connection must include the following points.

Arduino ground must connect to potentiometer pin 3. Arduino ground must connect to the LED cathode. Any measurement instrument ground must also connect to Arduino ground.

This requirement is absolute within the circuit. If the ground reference is missing, separated, or floating, the circuit behavior becomes unreliable.

3.10 Correct Circuit Scheme

The correct circuit scheme consists of two main parts. The first part is the analog input stage. The second part is the LED output stage.

In the analog input stage, the potentiometer is connected as a voltage divider. Pin 1 is connected to 5 V. Pin 3 is connected to ground. Pin 2 is connected to A0. This allows the Arduino to read a voltage that changes according to knob position.

In the LED output stage, Arduino pin D9 is connected to a 220 Ω resistor. The resistor is connected to the LED anode. The LED cathode is connected to ground. This allows the Arduino to control LED brightness through pulse width modulation while limiting current through the LED.

The correct circuit scheme can be summarized as follows.

Arduino 5 V connects to potentiometer pin 1. Arduino ground connects to potentiometer pin 3. Potentiometer pin 2 connects to Arduino A0. Arduino D9 connects to the 220 Ω resistor. The 220 Ω resistor connects to the LED anode. The LED cathode connects to Arduino ground.

This scheme is correct because it satisfies the requirements of voltage division, analog measurement, pulse width modulation output, current limitation, LED polarity, and common ground continuity.

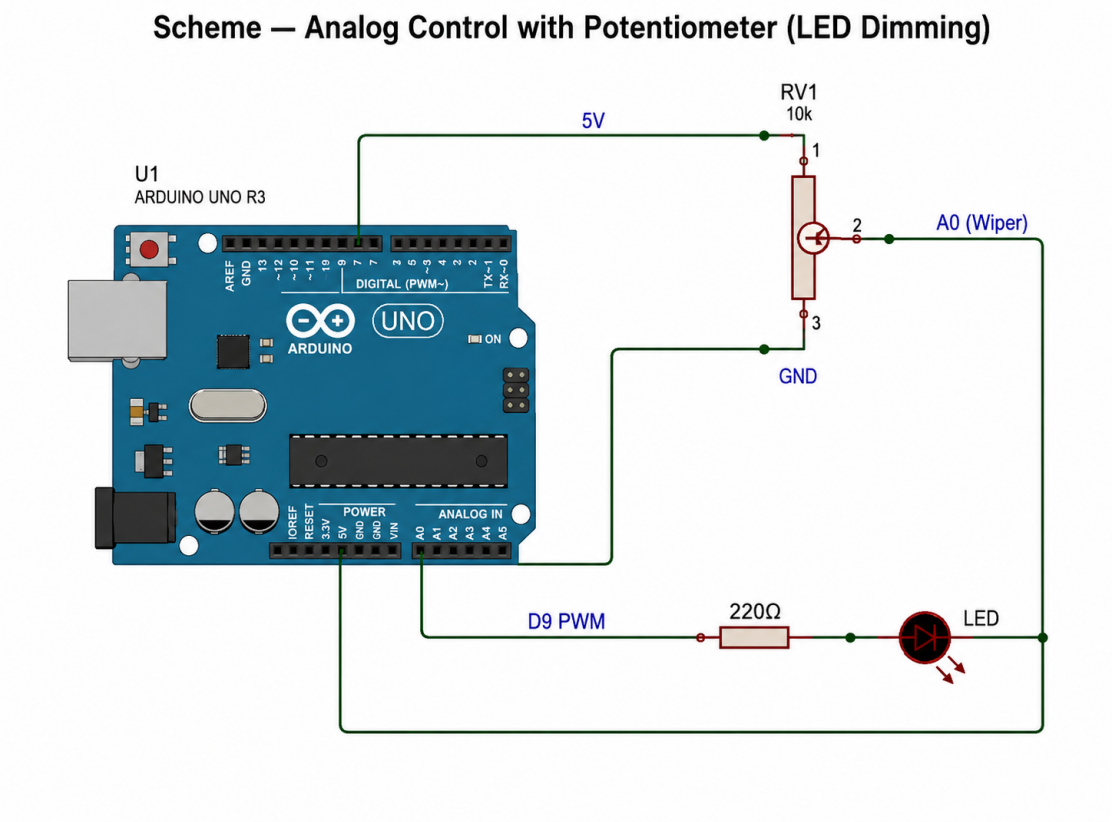

Figure 3.1: *Scheme - Analog Control with Potentiometer (LED Dimming)*

3.11 Proteus Schematic Layout Requirements

The Proteus schematic layout must be clear, readable, and logically arranged. The Arduino UNO R3 should be placed on the left side of the schematic. The potentiometer should be placed near the upper right side of the Arduino to represent the input section. The resistor and LED should be placed near the lower right side of the Arduino to represent the output section.

The wiring should be routed with clean right angle paths where possible. Wires should not cross unnecessarily. When wires must intersect, connection dots should be used only where electrical connection is intended. Signal labels should be added to improve readability. The 5 V line should be labeled clearly. The ground line should be labeled clearly. The A0 wiper signal should be labeled clearly. The D9 pulse width modulation line should be labeled clearly.

The schematic should include component reference labels and component values. The potentiometer should be labeled as RV1 with a value of 10 kΩ. The resistor should be labeled as R1 with a value of 220 Ω. The LED should be labeled as D1 or LED red. The Arduino should be labeled as U1 or Arduino UNO R3.

A high quality schematic must avoid visual ambiguity. The reader must be able to identify the input path, the output path, the ground path, and the control signal path without guessing.

3.12 Figure Placement for Scheme Chapter

Figures in the scheme chapter must be placed in an order that supports understanding. The first figure should show the full circuit. The second figure should focus on the potentiometer connection. The third figure should focus on the LED output connection.

This figure order is academically stronger than placing all details in one image only. The full scheme gives the complete system view. The potentiometer figure explains the analog input stage. The LED figure explains the controlled output stage. Together, these figures allow the reader to understand both the complete circuit and the critical local details.

Each figure must have a formal caption. The caption must state what the figure shows and why it is important. The figure must also be referenced in the text before or after placement.

3.12.1 Full Circuit Scheme Figure

The full circuit scheme figure should show the complete Proteus circuit, including the Arduino UNO R3, the 10 kΩ potentiometer, the 220 Ω resistor, the red LED, and the ground connections.

The figure must clearly show the following connections.

Arduino 5 V to potentiometer pin 1. Arduino ground to potentiometer pin 3. Potentiometer pin 2 to Arduino A0. Arduino D9 to the 220 Ω resistor. The 220 Ω resistor to the LED anode. The LED cathode to Arduino ground.

Suggested figure caption:

Figure 3.1. Complete Proteus circuit scheme for Arduino based potentiometer control of LED brightness.

This figure should be placed after the first explanation of the complete scheme so that the reader can immediately compare the written description with the visual circuit.

3.12.2 Potentiometer Connection Figure

The potentiometer connection figure should isolate or highlight the input section of the circuit. It should focus on the three potentiometer terminals and their connections to 5 V, ground, and A0.

The figure must show that pin 2 is the wiper terminal. This is the most important visual point because the most common mistake is connecting A0 to an outer pin. The figure should make clear that the outer pins define the voltage range while the center pin provides the variable signal.

Suggested figure caption:

Figure 3.2. Potentiometer voltage divider connection showing 5 V, ground, and A0 wiper signal.

This figure should be placed after the subsection on potentiometer wiring because it visually supports the explanation of pin 1, pin 2, and pin 3.

3.12.3 LED Output Connection Figure

The LED output connection figure should isolate or highlight the output section of the circuit. It should show Arduino D9 connected to the 220 Ω resistor, the resistor connected to the LED anode, and the LED cathode connected to ground.

The figure must make the series connection clear. The resistor and LED must appear in the same current path. The figure should also show the correct polarity of the LED. The anode must face the resistor side, and the cathode must face the ground side.

Suggested figure caption:

Figure 3.3. LED output circuit showing D9 pulse width modulation control through a 220 Ω current limiting resistor.

This figure should be placed after the subsection on LED output wiring because it visually confirms the protected current path.

3.13 Scheme Verification Checklist

The scheme must be verified before simulation. Verification ensures that the schematic is electrically correct and visually understandable.

The first verification point is the potentiometer supply connection. One outer terminal of the potentiometer must be connected to Arduino 5 V. The second verification point is the potentiometer ground connection. The other outer terminal must be connected to Arduino ground. The third verification point is the wiper connection. The center terminal of the potentiometer must be connected to Arduino A0.

The fourth verification point is the LED output path. Arduino D9 must connect to the 220 Ω resistor. The resistor must connect to the LED anode. The LED cathode must connect to ground. The fifth verification point is the common ground. The potentiometer ground and LED cathode ground must connect to the same Arduino ground reference.

The sixth verification point is the absence of a short circuit. There must be no direct connection between 5 V and ground. The seventh verification point is resistor placement. The resistor must be in series with the LED, not parallel to it and not disconnected from the LED branch. The eighth verification point is LED polarity. The anode must receive the signal through the resistor, and the cathode must return to ground.

A complete verification checklist is as follows.

Arduino 5 V is connected to potentiometer pin 1.

Arduino ground is connected to potentiometer pin 3.

Potentiometer pin 2 is connected to Arduino A0.

Arduino D9 is connected to the 220 Ω resistor.

The 220 Ω resistor is connected to the LED anode.

The LED cathode is connected to Arduino ground.

The potentiometer ground and LED ground share the same ground reference.

No direct short circuit exists between 5 V and ground.

The LED is not connected directly to D9 without a resistor.

The A0 wire is not connected to an outer potentiometer terminal.

The potentiometer wiper is not left floating.

The schematic labels are visible and unambiguous.

Component values are shown correctly as 10 kΩ and 220 Ω.

The circuit layout separates the input section from the output section.

All connection dots are placed only at true electrical junctions.

3.14 Common Scheme Mistakes to Avoid

Several common mistakes can prevent the circuit from working correctly. The first major mistake is connecting Arduino A0 to an outer potentiometer terminal. If A0 is connected to the 5 V outer terminal, the analog reading remains near 1023. If A0 is connected to the ground outer terminal, the analog reading remains near 0. In both cases, turning the potentiometer does not produce a variable reading.

The second mistake is leaving the wiper terminal unconnected. A floating analog input can produce unstable readings because the pin has no defined voltage. This may cause random LED brightness changes.

The third mistake is connecting the LED directly to D9 without a resistor. This is unsafe because it removes the current limiting element from the output path. The resistor is required to protect the LED and the Arduino output pin.

The fourth mistake is reversing the LED polarity. If the LED anode and cathode are reversed, the LED will not conduct current in the intended direction and will remain off.

The fifth mistake is failing to connect all ground points together. If the potentiometer ground is separated from Arduino ground, the analog input voltage may not be meaningful. If the LED cathode is not connected to Arduino ground, the LED current path remains incomplete.

The sixth mistake is using a non pulse width modulation pin for LED dimming while expecting analogWrite behavior. In this project, D9 is selected because it supports pulse width modulation output. Using an unsuitable pin may prevent smooth brightness control.

The seventh mistake is confusing the role of the potentiometer with the role of the Arduino. The potentiometer does not directly control the LED. It provides an analog input voltage. The Arduino reads this voltage, maps it to a pulse width modulation value, and then controls the LED through D9.

The eighth mistake is drawing an unclear schematic. Even if the electrical connections are correct, a confusing schematic weakens the academic quality of the project. Labels, component values, and connection points must be clear.

3.15 Chapter Summary

This chapter presented the complete scheme of the analog control circuit. The Arduino UNO R3 was identified as the controller, the 10 kΩ potentiometer as the analog input device, the 220 Ω resistor as the current limiting component, and the red LED as the visual output device. The chapter explained that the potentiometer must be wired as a voltage divider, with one outer terminal connected to 5 V, the other outer terminal connected to ground, and the center wiper terminal connected to A0.

The chapter also explained the LED output path. Arduino pin D9 provides the pulse width modulation signal. This signal passes through the 220 Ω resistor before reaching the LED anode. The LED cathode returns to ground, completing the current path. The resistor limits current and protects the circuit.

The common ground requirement was established as a necessary condition for correct operation. The Arduino, potentiometer, and LED branch must share the same ground reference. The chapter also defined the required Proteus components, component selection logic, schematic layout requirements, figure placement, verification checklist, and common mistakes to avoid.

The correct scheme can therefore be summarized in one complete statement. The circuit works because the potentiometer converts mechanical rotation into a variable voltage at A0, the Arduino converts that voltage into a pulse width modulation output at D9, and the LED brightness changes through a protected resistor controlled output path.

4
Simulation

4.1 Introduction to Simulation

Simulation is a necessary stage in the development of the analog control circuit because it allows the circuit behavior to be examined before physical construction. In this project, the simulation verifies that a potentiometer can produce a variable analog voltage, that the Arduino UNO can read this voltage through analog input A0, and that the resulting value can be converted into a pulse width modulation signal on digital pin D9 to control the brightness of an LED.

The simulation chapter therefore connects the theoretical calculations from Chapter 2 with the practical circuit arrangement from Chapter 3. The calculations predict the expected voltage, analog reading, pulse width modulation value, and LED brightness level. The simulation tests whether these predicted values appear correctly when the circuit is operated in Proteus.

The simulation is not included only to show that the LED turns on. Its main academic purpose is to verify the full control chain of the system. The chain begins with mechanical rotation of the potentiometer, continues through voltage variation at the wiper terminal, passes into the Arduino analog input, is converted into a digital numerical value, is mapped into a pulse width modulation output value, and finally appears as a visible brightness change in the LED.

A correct simulation must therefore prove four main points. First, the potentiometer must create a voltage that varies between approximately 0 V and 5 V. Second, the Arduino analog input must convert this voltage into a value between 0 and 1023. Third, the program must map this analog value into a pulse width modulation value between 0 and 255. Fourth, the LED brightness must change according to the pulse width modulation value.

4.2 Purpose of Proteus Simulation

The purpose of the Proteus simulation is to test the correctness, stability, and expected operation of the analog control circuit before implementation on real hardware. Proteus provides a controlled environment where the circuit can be observed without the

risk of damaging physical components through incorrect wiring, missing resistance, reversed LED polarity, or incorrect pin selection.

In this project, Proteus simulation has five specific purposes.

First, it verifies the wiring of the potentiometer. The simulation confirms that one outer potentiometer terminal is connected to 5 V, the other outer terminal is connected to ground, and the center wiper terminal is connected to Arduino analog input A0.

Second, it verifies the analog input behavior. When the potentiometer is adjusted, the voltage at A0 must change smoothly. If A0 remains fixed at 0 V or 5 V, the circuit is not functioning as an analog control system.

Third, it verifies the Arduino program. The program must read the analog input, convert the analog value into a pulse width modulation value, and send that value to digital pin D9.

Fourth, it verifies the LED output behavior. The LED must become dim at low potentiometer positions, medium bright at middle positions, and fully bright at high positions.

Fifth, it provides evidence for the academic report. The simulation figures, readings, and result table support the claim that the circuit operates according to the theoretical design.

4.3 Simulation Environment

The simulation environment consists of Proteus design software, an Arduino UNO R3 model, a 10 kΩ potentiometer, a 220 Ω resistor, a red LED, ground terminals, connecting wires, and the compiled Arduino program file. The Arduino program must be prepared externally and then loaded into the Arduino UNO model inside Proteus.

The circuit must be constructed in Proteus using the same wiring logic described in the scheme chapter. The Arduino UNO is placed as the main controller. The potentiometer is placed as the analog input device. The LED and resistor are placed as the output branch. The ground terminals must be connected consistently so that the potentiometer ground, LED ground, and Arduino ground share the same electrical reference.

The simulation environment should include measurement tools if available. A direct current voltmeter can be connected between A0 and ground to observe the analog voltage. An oscilloscope can be connected to D9 and ground to observe the pulse width modulation waveform. These instruments are not required for the LED to operate, but they make the simulation evidence stronger and more measurable.

The simulation must be arranged so that the circuit is readable, traceable, and suitable for inclusion in an academic report. Wires should be routed clearly. Component names and values should be visible. The potentiometer should be labeled as 10 kΩ. The resistor should be labeled as 220 Ω. The LED should be labeled as a red LED. The Arduino pins A0, D9, 5 V, and ground should be clearly identifiable.

4.4 Arduino Program Preparation

The Arduino program is prepared before running the Proteus simulation. The program must define the potentiometer input pin, define the LED output pin, read the analog value from A0, convert that value into a pulse width modulation value, and write the converted value to D9.

The program must use A0 as the analog input because the potentiometer wiper is connected to A0. It must use D9 as the output pin because D9 supports pulse width modulation on the Arduino UNO. The use of a pulse width modulation pin is essential because LED dimming requires a variable duty cycle rather than a simple on or off digital output.

The program preparation process should include the following operations. The correct board type must be selected as Arduino UNO. The code must be checked for syntax errors. The program must be compiled successfully. The compiled output file must then be exported so it can be loaded into the Arduino UNO component inside Proteus.

A correct program must not write a fixed value to the LED pin. If the program writes only 0, the LED will remain off. If the program writes only 255, the LED will remain fully bright. The program must continuously read the changing potentiometer value and continuously update the output value.

4.5 Arduino Code Used in Simulation

The following Arduino code is used for the simulation. The code reads the potentiometer voltage from A0, converts the analog reading into a pulse width modulation value, and sends the result to digital pin D9.

Listing 4.1: *Arduino code used in simulation*

```
const int potPin = A0;
const int ledPin = 9;
void setup() {
pinMode(ledPin, OUTPUT);
}

    void loop() {
    int analogValue = analogRead(potPin);
    int pwmValue = map(analogValue, 0, 1023, 0, 255);
    analogWrite(ledPin, pwmValue);
    }
```

The variable potPin identifies A0 as the input pin. The variable ledPin identifies D9 as the output pin. The function analogRead reads the voltage at A0 and returns a value from 0 to 1023. The function map converts this value into the output range from 0 to 255. The function analogWrite applies the resulting pulse width modulation value to D9.

The program must run continuously because the potentiometer position can change at any moment. Each loop cycle updates the LED brightness according to the latest

analog reading.

4.6 HEX File Generation

Proteus does not directly execute the written Arduino source code. It requires the compiled machine level program file that represents the code in a format suitable for the simulated microcontroller. This file is commonly generated after the Arduino program is compiled.

The HEX file generation process begins by compiling the Arduino code. If the code has no syntax error and the correct board is selected, the Arduino development environment creates the compiled program output. This compiled output is then used as the program file for the Arduino UNO component in Proteus.

The purpose of the HEX file is to give the simulated Arduino the same logical behavior that a real Arduino would have after uploading the program. Without the HEX file, the Proteus Arduino model may appear physically connected, but it will not execute the intended control logic. In that case, the LED may remain unchanged, and the simulation will not represent the actual project.

The HEX file therefore forms the bridge between software logic and circuit simulation. The circuit provides the electrical structure, while the HEX file provides the programmed behavior of the controller.

4.7 Loading the HEX File into Proteus

After the HEX file has been generated, it must be loaded into the Arduino UNO component inside Proteus. This is done by opening the properties of the Arduino UNO model and selecting the compiled program file in the program file field.

Once the file is selected, Proteus associates the Arduino model with the compiled code. During simulation, the Arduino model then executes the program as if the code had been uploaded to a physical Arduino UNO board.

Correct loading of the HEX file must be verified before analyzing the circuit behavior. If the HEX file is missing, incorrectly selected, or generated from the wrong code, the simulation result will not be valid. A circuit that is wired correctly can still fail if the Arduino model does not contain the correct program.

The report should mention that the HEX file was loaded into the Arduino UNO model before starting the simulation. This confirms that the observed LED behavior is caused by the intended program and not by an incomplete simulation setup.

4.8 Simulation Circuit Setup

The simulation circuit setup follows the exact scheme defined in the circuit chapter. The Arduino UNO is used as the control unit. The 10 kΩ potentiometer is used as the analog input device. The red LED is used as the visible output device. The 220 Ω resistor is used to limit the LED current.

The potentiometer is wired as a voltage divider. One outer terminal is connected to the Arduino 5 V pin. The opposite outer terminal is connected to Arduino ground.

The center terminal, which is the wiper, is connected to Arduino analog input A0. This connection is essential because the wiper produces the variable voltage that represents the knob position.

The LED branch is connected to Arduino digital pin D9. Pin D9 is connected to one side of the 220 Ω resistor. The other side of the resistor is connected to the LED anode. The LED cathode is connected to Arduino ground. This arrangement ensures that the LED current is limited and that the LED brightness can be controlled through pulse width modulation.

The circuit setup must maintain one common ground reference. The ground of the potentiometer and the ground of the LED must both return to Arduino ground. If the grounds are not common, the analog reading and the output response may become invalid or unstable.

4.9 Initial Simulation Conditions

Before running the simulation, the initial conditions must be defined. The Arduino UNO must be powered through its simulated supply system. The potentiometer value must be set to 10 kΩ. The resistor value must be set to 220 Ω. The LED must be correctly oriented, with its anode connected to the resistor and its cathode connected to ground.

The initial potentiometer position may be set to 0 percent, 50 percent, or another known value. For systematic testing, it is better to begin from 0 percent because this represents the minimum analog voltage and should produce the minimum LED brightness. The potentiometer can then be increased step by step to 25 percent, 50 percent, 75 percent, and 100 percent.

The expected voltage at A0 depends directly on the potentiometer position. If the position is represented by p, then the theoretical voltage is:

$$V_A0 = 5V \times p$$

The expected analog reading is:

$$ADC \approx V_A0 \div 5V \times 1023$$

The expected pulse width modulation value is:

$$PWM \approx ADC \times 255 \div 1023$$

The simulation should be started only after confirming that the wiring, component values, Arduino program file, and ground connections are correct.

4.10 Potentiometer Position Testing

Potentiometer position testing is the main experimental procedure in the simulation chapter. The potentiometer is adjusted to different positions, and the corresponding A0 voltage, analog reading, pulse width modulation value, and LED brightness are observed.

The purpose of testing several positions is to prove that the system responds gradually rather than only switching between off and on. A correct analog control system must show proportional behavior. As the potentiometer position increases, the A0 voltage should increase, the ADC value should increase, the pulse width modulation value should increase, and the LED brightness should increase.

The selected test positions are 0 percent, 25 percent, 50 percent, 75 percent, and 100 percent. These positions cover the complete operating range of the circuit and provide enough data points to compare theory with simulation.

4.10.1 Potentiometer at 0 Percent

When the potentiometer is at 0 percent, the wiper is electrically closest to ground. The voltage at A0 is expected to be approximately 0 V.

The expected analog reading is:

$$ADC \approx 0V \div 5V \times 1023$$

$$ADC \approx 0$$

The expected pulse width modulation value is:

$$PWM \approx 0 \times 255 \div 1023$$

$$PWM \approx 0$$

At this position, the LED should be off or nearly off. This result confirms that the minimum potentiometer position produces the minimum output brightness.

4.10.2 Potentiometer at 25 Percent

When the potentiometer is at 25 percent, the wiper voltage is expected to be one quarter of the supply voltage.

The expected A0 voltage is:

$$V_A0 = 5V \times 0.25$$

$$V_A0 = 1.25V$$

The expected analog reading is:

$$ADC \approx 1.25V \div 5V \times 1023$$

$$ADC \approx 255.75$$

$$ADC \approx 256$$

The expected pulse width modulation value is:

$$PWM \approx 256 \times 255 \div 1023$$

$$PWM \approx 64$$

At this position, the LED should appear dim. The LED is not fully off because D9 is producing a small duty cycle. The brightness should be visibly lower than the brightness at 50 percent, 75 percent, and 100 percent.

4.10.3 Potentiometer at 50 Percent

When the potentiometer is at 50 percent, the wiper voltage is expected to be half of the supply voltage.

The expected A0 voltage is:

$$V_A0 = 5V \times 0.50$$

$$V_A0 = 2.50V$$

The expected analog reading is:

$$ADC \approx 2.50V \div 5V \times 1023$$

$$ADC \approx 511.5$$

$$ADC \approx 512$$

The expected pulse width modulation value is:

$$PWM \approx 512 \times 255 \div 1023$$

$$PWM \approx 128$$

At this position, the LED should show medium brightness. This is a central test point because it verifies that the circuit can produce an output between the minimum and maximum brightness states.

4.10.4 Potentiometer at 75 Percent

When the potentiometer is at 75 percent, the wiper voltage is expected to be three quarters of the supply voltage.

The expected A0 voltage is:

$$V_A0 = 5V \times 0.75$$

$$V_A0 = 3.75V$$

The expected analog reading is:

$$ADC \approx 3.75V \div 5V \times 1023$$

$$ADC \approx 767.25$$

$$ADC \approx 767$$

The expected pulse width modulation value is:

$$PWM \approx 767 \times 255 \div 1023$$

$$PWM \approx 191$$

At this position, the LED should appear bright. The LED should be clearly brighter than at 50 percent but still slightly below the maximum brightness observed at 100 percent.

4.10.5 Potentiometer at 100 Percent

When the potentiometer is at 100 percent, the wiper is electrically closest to the 5 V terminal. The voltage at A0 is expected to be approximately 5 V.

The expected analog reading is:

$$ADC \approx 5V \div 5V \times 1023$$

$$ADC \approx 1023$$

The expected pulse width modulation value is:

$$PWM \approx 1023 \times 255 \div 1023$$

$$PWM \approx 255$$

At this position, the LED should appear fully bright. This verifies the maximum control state of the circuit.

4.11 A0 Voltage Observation

The A0 voltage observation confirms whether the potentiometer is functioning as a voltage divider. The voltage at A0 should vary according to the position of the potentiometer wiper. If the potentiometer is connected correctly, A0 should measure

approximately 0 V at the minimum position and approximately 5 V at the maximum position.

A voltmeter may be placed between A0 and ground in Proteus. The positive terminal of the voltmeter should connect to A0, and the negative terminal should connect to ground. As the potentiometer is rotated, the voltmeter reading should increase or decrease smoothly.

The expected voltage values are:

Table 4.1: *Potentiometer position and expected voltage*

Potentiometer Position	Expected A0 Voltage
0 percent	0 V
25 percent	1.25 V
50 percent	2.50 V
75 percent	3.75 V
100 percent	5.00 V

If the measured voltage does not change when the potentiometer is adjusted, the most likely cause is that A0 is connected to an outer potentiometer terminal instead of the center wiper terminal.

4.12 ADC Reading Observation

The ADC reading observation verifies that the Arduino correctly converts the analog voltage into a digital numerical value. The Arduino UNO analog input converts the voltage range from 0 V to 5 V into values from 0 to 1023.

The expected relationship is:

$$ADC \approx V_A0 \div 5V \times 1023$$

The expected values are:

Table 4.2: *A0 voltage and expected ADC reading*

Potentiometer Position	A0 Voltage	Expected ADC Reading
0 percent	0 V	0
25 percent	1.25 V	256
50 percent	2.50 V	512
75 percent	3.75 V	767
100 percent	5.00 V	1023

The ADC value may not always appear as a perfectly fixed number in simulation because of rounding, component modeling, and numerical calculation inside the simulation environment. A small difference is acceptable if the value remains close to the theoretical result and follows the correct increasing pattern.

4.13 PWM Output Observation

The PWM output observation verifies that the Arduino program correctly converts the analog reading into a pulse width modulation output value on D9. The code maps the ADC range from 0 to 1023 into a PWM range from 0 to 255.

The expected relationship is:

$$PWM \approx ADC \times 255 \div 1023$$

The expected values are:

Table 4.3: *Expected ADC reading and PWM value*

Potentiometer Position	Expected ADC Reading	Expected PWM Value
0 percent	0	0
25 percent	256	64
50 percent	512	128
75 percent	767	191
100 percent	1023	255

The PWM waveform can be observed with an oscilloscope in Proteus. At lower potentiometer positions, the high portion of the waveform should be short. At higher potentiometer positions, the high portion should become longer. The waveform frequency remains controlled by the Arduino timer behavior, while the duty cycle changes according to the mapped value.

The LED brightness changes because the average delivered energy changes with the duty cycle. At a low duty cycle, the LED receives current for a small portion of each cycle and appears dim. At a high duty cycle, the LED receives current for a larger portion of each cycle and appears brighter.

4.14 LED Brightness Observation

The LED brightness observation provides the visible confirmation of the complete circuit operation.
The LED brightness should correspond to the potentiometer position. A low potentiometer position should produce low brightness. A middle position should produce medium brightness. A high position should produce high brightness.

The LED current during the on portion of the pulse is limited by the 220 Ω resistor. Assuming a 5 V output and an approximate red LED forward voltage of 2 V, the peak current during the on state is:

$$I_peak \approx 5V \text{minus} 2V \text{, divided by} 220\Omega$$

$$I_peak \approx 3V \div 220\Omega$$

$$I_peak \approx 0.0136A$$

$$I_peak \approx 13.6mA$$

The average LED current depends on the PWM duty cycle. The approximate average current can be estimated as:

$$I_average \approx I_peak \times dutycycle$$

Therefore, the approximate average current values are:

Table 4.4: *PWM duty cycle and approximate average LED current*

Potentiometer Position	PWM Duty Cycle	Approximate Average LED Current
0 percent	0 percent	0 mA
25 percent	25 percent	3.4 mA
50 percent	50 percent	6.8 mA
75 percent	75 percent	10.2 mA
100 percent	100 percent	13.6 mA

These values explain why the LED brightness increases as the potentiometer is turned upward. The simulation should visually support this relationship.

4.15 Simulation Result Table

The simulation result table organizes the expected values and observed values in one academic form. The table should include the potentiometer position, expected A0 voltage, expected ADC reading, expected PWM value, expected LED behavior, and observed simulation result.

Table 4.5: *Simulation result table*

Potentiometer Position	Expected A0 Voltage	Expected ADC Reading	Expected PWM Value	Expected LED Brightness
0 percent	0 V	0	0	Off
25 percent	1.25 V	256	64	Dim
50 percent	2.50 V	512	128	Medium brightness
75 percent	3.75 V	767	191	Bright
100 percent	5.00 V	1023	255	Fully bright

The report may add an additional column for observed values after the simulation is performed. If the observed values are close to the expected values and follow the same increasing pattern, the simulation can be considered successful.

4.16 Comparison Between Calculation and Simulation

The comparison between calculation and simulation determines whether the simulated circuit behaves according to the theoretical design. The calculation chapter predicts the

values of A0 voltage, ADC reading, PWM output, and LED brightness. The simulation chapter tests whether these predicted values are reproduced in the Proteus environment.

A correct comparison should show that the A0 voltage increases in proportion to the potentiometer position. It should also show that the ADC reading increases as the voltage increases. The PWM value should increase as the ADC value increases. Finally, the LED brightness should increase as the PWM value increases.

The relationship can be summarized as follows:

Higher potentiometer position produces higher A0 voltage

Higher A0 voltage produces higher ADC reading

Higher ADC reading produces higher PWM value

Higher PWM value produces higher LED brightness

Minor differences between calculated and simulated values are acceptable because rounding can occur during analog to digital conversion and during the mapping from 0 to 1023 into 0 to 255. However, the direction and proportional pattern must remain correct. If the values do not increase in the correct order, the simulation result is not valid.

4.17 Simulation Accuracy Discussion

The accuracy of the simulation depends on correct component values, correct wiring, correct code, and correct interpretation of the results. The most important accuracy condition is the correct connection of the potentiometer wiper to A0. If the wiper is not connected to A0, the analog voltage will not represent the potentiometer position.

The second accuracy condition is the correct value of the potentiometer. A 10 kΩ potentiometer is appropriate because it provides a stable voltage divider without drawing excessive current from the Arduino 5 V supply. If the resistance is extremely low, unnecessary current may flow through the potentiometer. If the resistance is extremely high, the analog input may become more sensitive to noise.

The third accuracy condition is the correct resistor value in the LED branch. The 220 Ω resistor limits current to a safe range for the LED and Arduino output pin. Without this resistor, the LED branch would not be electrically safe.

The fourth accuracy condition is the correct use of a PWM pin. Since the LED brightness is controlled through analogWrite, the output pin must support pulse width modulation. D9 is suitable for this purpose on the Arduino UNO.

The fifth accuracy condition is correct visual interpretation. LED brightness in simulation is an approximation of real visual brightness. Human perception of brightness is not perfectly linear, and the simulated LED display may not perfectly represent physical light intensity. Therefore, the main evidence should include not only LED appearance but also measured A0 voltage and PWM behavior.

4.18 Figure Placement for Simulation Chapter

Figures in the simulation chapter should be placed where they directly support the explanation. Each figure must have a clear title and a short academic caption. The

figure should not be decorative. It must provide evidence for a specific claim in the chapter.

The recommended figure sequence is:

Figure 4.1 should show the simulation at low brightness.

Figure 4.2 should show the simulation at medium brightness.

Figure 4.3 should show the simulation at high brightness.

Figure 4.4 should show the PWM output observation.

Figure 4.5 should show the A0 voltage measurement.

This figure order is appropriate because it moves from visible circuit behavior to measured electrical evidence.

4.18.1 Simulation at Low Brightness

This figure should show the potentiometer set near the low position, such as 25 percent. The expected A0 voltage is approximately 1.25 V, the expected ADC reading is approximately 256, and the expected PWM value is approximately 64.

The LED should appear dim. The purpose of this figure is to prove that a low potentiometer position produces a low PWM duty cycle and a low brightness output.

Suggested caption:

Figure 4.1. Proteus simulation of the circuit at low potentiometer position showing dim LED brightness.

4.18.2 Simulation at Medium Brightness

This figure should show the potentiometer set near the middle position, such as 50 percent. The expected A0 voltage is approximately 2.50 V, the expected ADC reading is approximately 512, and the expected PWM value is approximately 128.

The LED should appear at medium brightness. The purpose of this figure is to prove that the circuit can operate between minimum and maximum output states.

Suggested caption:

Figure 4.2. Proteus simulation of the circuit at middle potentiometer position showing medium LED brightness.

4.18.3 Simulation at High Brightness

This figure should show the potentiometer set near the high position, such as 75 percent or 100 percent. At 75 percent, the expected A0 voltage is approximately 3.75 V, the expected ADC reading is approximately 767, and the expected PWM value is approximately 191. At 100 percent, the expected A0 voltage is approximately 5.00 V, the expected ADC reading is approximately 1023, and the expected PWM value is 255.

The LED should appear bright or fully bright. The purpose of this figure is to confirm that a high potentiometer position produces a high duty cycle and a strong LED output.

Suggested caption:

Figure 4.3. Proteus simulation of the circuit at high potentiometer position showing bright LED output.

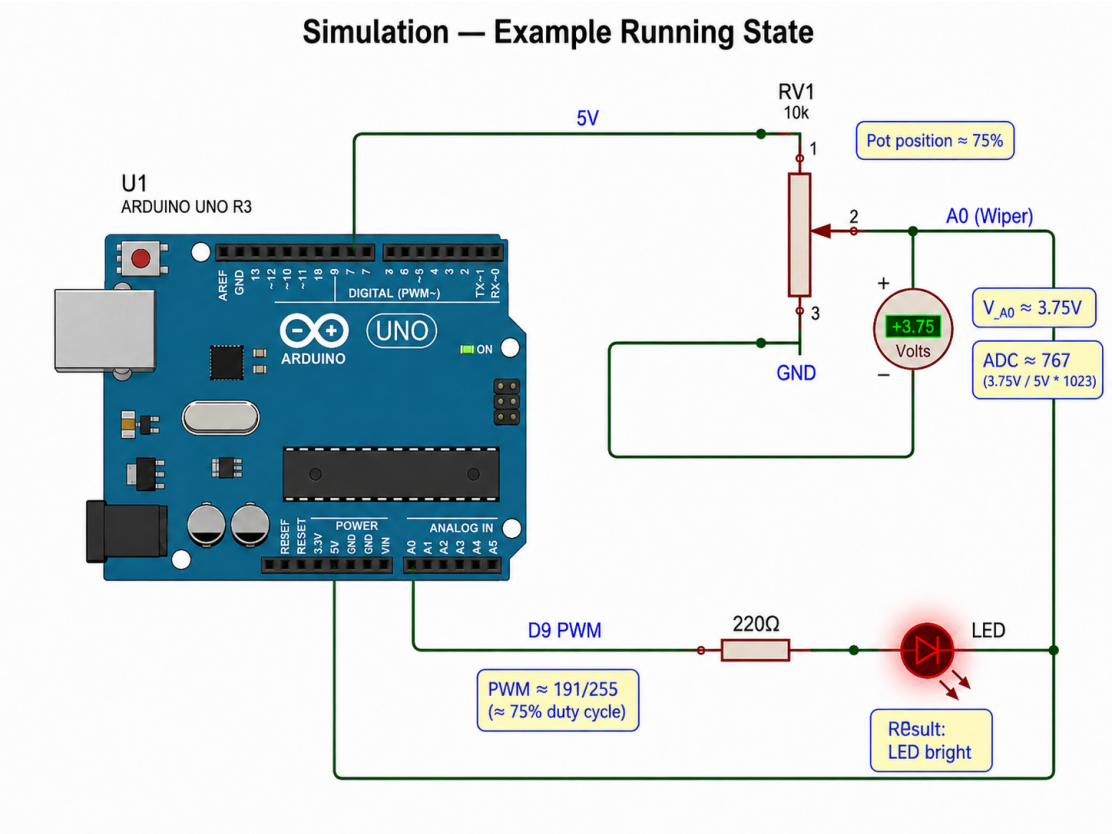

Figure 4.1: *Simulation - Example Running State*

4.18.4 PWM Output Observation Figure

This figure should show the PWM signal produced by Arduino digital pin D9. The oscilloscope or signal measurement tool should be connected between D9 and ground. The waveform should show repeated switching between low and high states.

The important feature is the duty cycle. At low potentiometer positions, the high portion of the waveform should be shorter. At high potentiometer positions, the high portion should be longer.

Suggested caption:

Figure 4.4. PWM waveform observed at Arduino digital pin D9 during LED brightness control.

4.18.5 A0 Voltage Measurement Figure

This figure should show the measured voltage at Arduino analog input A0. A voltmeter should be connected with its positive terminal at A0 and its negative terminal at ground.

The figure should confirm that the potentiometer wiper produces a variable voltage. At 50 percent position, the reading should be close to 2.50 V. At 75 percent position, the reading should be close to 3.75 V.

Suggested caption:

Figure 4.5. Measurement of the variable analog voltage at Arduino input A0.

4.19 Simulation Validation Checklist

The simulation is considered valid only if all essential conditions are satisfied. The validation checklist should be included at the end of the simulation chapter to confirm that the simulation result is complete and reliable.

The checklist is as follows.

Table 4.6: *Simulation validation checklist*

Validation Point	Required Condition	Status
Arduino model included	Arduino UNO R3 is placed in Proteus	To be checked
Program loaded	Correct HEX file is loaded into Arduino model	To be checked
Potentiometer value	Potentiometer is set to 10 kΩ	To be checked
Potentiometer supply	One outer terminal is connected to 5 V	To be checked
Potentiometer ground	Other outer terminal is connected to ground	To be checked
Wiper connection	Center terminal is connected to A0	To be checked
LED resistor	220 Ω resistor is in series with the LED	To be checked
LED polarity	LED anode faces resistor and cathode returns to ground	To be checked
PWM pin	D9 is used as the output pin	To be checked
Common ground	Potentiometer and LED share Arduino ground	To be checked
A0 voltage	Voltage changes from 0 V to 5 V range	To be checked
ADC behavior	ADC value changes from 0 to 1023 range	To be checked
PWM behavior	PWM value changes from 0 to 255 range	To be checked
LED response	LED brightness changes with potentiometer position	To be checked

A simulation that satisfies all validation points can be accepted as successful. If any point fails, the circuit must be corrected before the simulation result is used as evidence in the report.

4.20 Chapter Summary

This chapter presented the simulation stage of the analog control project. The simulation was designed to verify the relationship between potentiometer position, A0 voltage, ADC reading, PWM output, and LED brightness. The Proteus environment was used to test the circuit before physical implementation.

The chapter explained the preparation of the Arduino program, the generation and loading of the HEX file, the simulation circuit setup, and the required initial conditions. The potentiometer was tested at 0 percent, 25 percent, 50 percent, 75 percent, and 100

percent positions. For each position, the expected voltage, ADC reading, PWM value, and LED brightness were identified.

The simulation confirms the central operating principle of the project. When the potentiometer position increases, the voltage at A0 increases. When the voltage at A0 increases, the ADC reading increases. When the ADC reading increases, the PWM value on D9 increases. When the PWM value increases, the LED becomes brighter.

The chapter also established the required figure placements and validation checklist. These elements ensure that the simulation chapter is not only descriptive but also measurable, verifiable, and academically complete.

5 Troubleshooting

5.1 Introduction to Troubleshooting

Troubleshooting is an essential stage in the development and verification of an embedded electronic system because it determines whether the designed circuit performs according to its theoretical and simulated expectations. In the present project, the system depends on the correct interaction between the Arduino UNO R3, the 10k potentiometer, the analog input pin A0, the PWM output pin D9, the 220Ω current limiting resistor, and the red LED. Any incorrect connection, incorrect code statement, incorrect component orientation, missing ground reference, or incomplete simulation setting can prevent the circuit from operating correctly.

The purpose of this chapter is to identify, explain, classify, and correct the most important faults that may occur in the analog control circuit. The chapter does not treat troubleshooting as a random trial process. Instead, it treats troubleshooting as a structured technical investigation in which every visible symptom is connected to a possible electrical, logical, or simulation based cause. This approach allows the user to move from observation to diagnosis and from diagnosis to correction in a controlled and academically defensible way.

The circuit is expected to allow the brightness of the LED to change when the potentiometer knob is rotated. This expected behavior depends on one critical principle: the potentiometer must operate as a voltage divider, and the center wiper terminal must provide the variable voltage to Arduino analog input A0. If the Arduino receives a changing voltage at A0, the program can convert that analog reading into a PWM value and apply it to digital pin D9. If the Arduino does not receive a changing voltage, the output brightness will not change correctly, even if the LED branch itself is properly connected.

Therefore, troubleshooting in this project must examine the complete path of operation. The investigation begins at the input side, continues through the analog reading and PWM control logic, and ends at the LED output side. A valid fault analysis must consider the potentiometer connection, the analog input reading, the program logic, the PWM output pin, the LED polarity, the resistor placement, the ground connection, and the Proteus simulation setup.

5.2 Purpose of Troubleshooting

The purpose of troubleshooting is to ensure that the final circuit is electrically correct, logically correct, and operationally reliable. In an academic report, troubleshooting is not included only to describe possible mistakes. It is included to prove that the designer understands how the system behaves when it is correct and how the same system fails when one of its structural requirements is violated.

The first purpose is to verify the correctness of the input stage. The input stage consists of the 10k potentiometer connected between 5V and GND, with its center wiper connected to Arduino A0. This stage must generate a voltage between 0V and 5V. When the potentiometer position changes, the voltage at A0 must also change. If this voltage does not change, the fault must be located before the PWM output stage is blamed.

The second purpose is to verify the correctness of the control stage. The Arduino must read the analog voltage through analogRead(A0). The expected reading range is from 0 to 1023. This value must then be mapped to a PWM range from 0 to 255. If the analog reading is correct but the LED brightness does not change, the fault may exist in the program logic, the selected output pin, or the LED branch.

The third purpose is to verify the correctness of the output stage. The LED must be connected in series with a 220Ω resistor. The resistor must limit the current, and the LED must be connected with correct polarity. The Arduino D9 pin must provide the PWM signal to the resistor and LED path. The LED cathode must return to GND. If any of these conditions is not satisfied, the LED may remain off, remain fully bright, flicker, or operate unsafely.

The fourth purpose is to verify the correctness of the simulation environment. In Proteus, a correct circuit may still fail if the Arduino program file is not loaded, if the simulation is not started correctly, if the wrong component model is selected, or if the ground reference is missing. Simulation faults must therefore be separated from actual circuit design faults.

Troubleshooting therefore protects the academic quality of the project by confirming that the final result is not accidental. A correct result must be repeatable, explainable, and traceable to a verified set of electrical and logical conditions.

5.3 Correct Behavior Reference

Before faults can be diagnosed, the correct behavior of the system must be clearly defined. The correct behavior reference is the standard against which all abnormal behavior is compared. Without a correct behavior reference, troubleshooting becomes uncertain because there is no fixed baseline for identifying whether the observed result is normal or faulty.

In the correct circuit, the potentiometer is connected as a voltage divider. One outer terminal is connected to 5V, the other outer terminal is connected to GND, and the center wiper terminal is connected to Arduino A0. When the knob is rotated toward the GND side, the voltage at A0 approaches 0V. When the knob is rotated toward the 5V side, the voltage at A0 approaches 5V. At intermediate positions, the voltage at A0 takes intermediate values.

The relationship between potentiometer position and analog input voltage may be expressed as follows:

$$V_A0 = 5V \times p$$

In this expression, p represents the normalized position of the potentiometer, where p = 0 corresponds to the minimum position and p = 1 corresponds to the maximum position. If the potentiometer is at the midpoint, then p = 0.5, and the expected voltage at A0 is:

$$V_A0 = 5V \times 0.5 = 2.5V$$

The Arduino analog input then converts this voltage into a digital value. For a 10 bit analog to digital conversion range, the expected analog reading is:

$$ADC = (V_A0 \div 5V) \times 1023$$

At V_A0 = 2.5V, the expected ADC value is:

$$ADC = (2.5V \div 5V) \times 1023 = 511.5$$

This value is normally interpreted as approximately 512. The program then maps this value to a PWM range between 0 and 255:

$$PWM = ADC \div 4$$

At an ADC value of approximately 512, the expected PWM value is:

$$PWM = 512 \div 4 = 128$$

This means that the LED should appear at approximately medium brightness. At low potentiometer positions, the LED should appear dim or off. At high potentiometer positions, the LED should appear bright. The correct behavior is therefore not only that the LED turns on, but that its brightness changes progressively as the potentiometer position changes.

A correct system must satisfy all of the following conditions. The A0 voltage must change smoothly from approximately 0V to approximately 5V. The ADC value must change from approximately 0 to approximately 1023. The PWM value must change from approximately 0 to approximately 255. The LED brightness must increase as the PWM value increases. The LED must not remain fixed while the potentiometer is being rotated. The resistor must remain in series with the LED, and the LED must return to the same ground reference as the Arduino.

5.4 Main Fault Case: Wrong Potentiometer Connection

The main fault case in this project is the incorrect connection of the potentiometer. This fault is especially important because the potentiometer has three terminals, and beginners often connect the Arduino analog input to an outer terminal instead of the

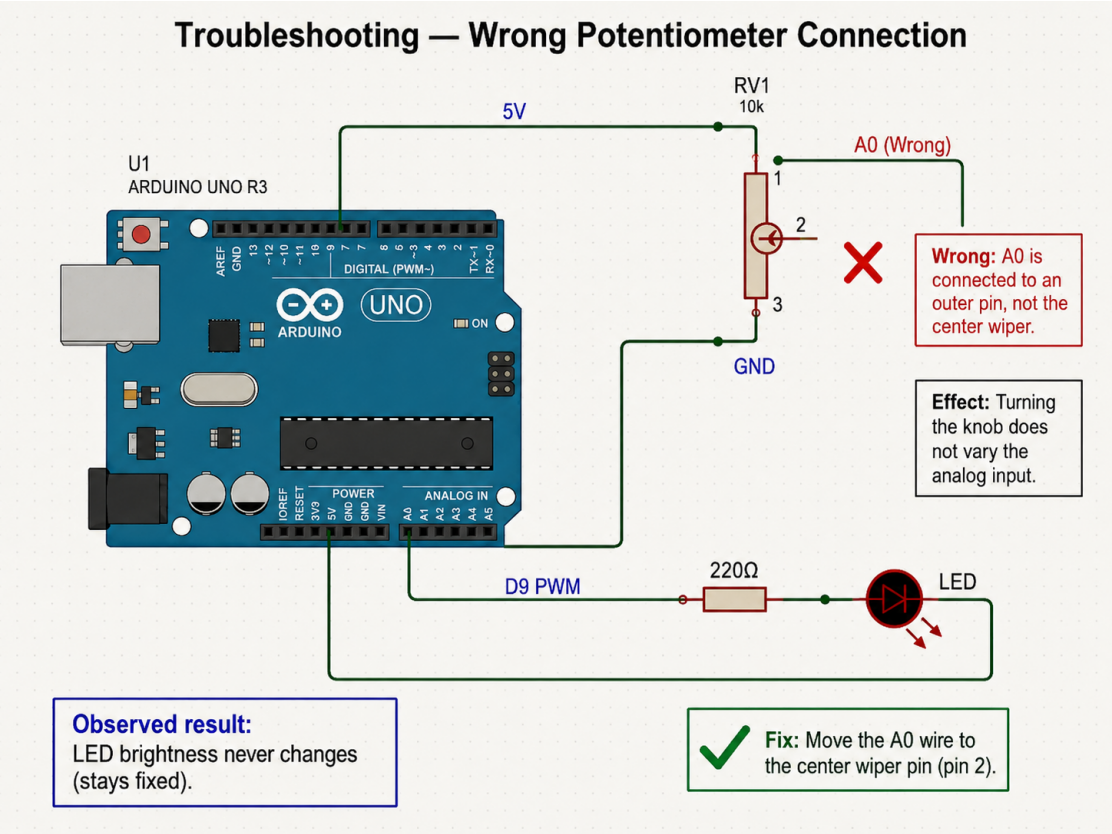

Figure 5.1: *Troubleshooting - Wrong Potentiometer Connection*

center wiper terminal. The error may appear small in the schematic, but it completely changes the behavior of the circuit.

The potentiometer must be understood as a three terminal device. The two outer terminals define the full resistance track. When these terminals are connected to 5V and GND, they create the two voltage limits of the divider. The center terminal, called the wiper, moves along the resistive track and produces the variable voltage. The Arduino must read this variable voltage from the wiper. If A0 is not connected to the wiper, the Arduino cannot detect the rotation of the potentiometer correctly.

In the wrong connection case, A0 may be connected to an outer terminal. If A0 is connected to the outer terminal connected to 5V, the analog reading will remain close to 1023. The program will then produce a PWM value close to 255, and the LED will remain fully bright. If A0 is connected to the outer terminal connected to GND, the analog reading will remain close to 0. The program will then produce a PWM value close to 0, and the LED will remain off. In both cases, rotating the potentiometer will not produce the expected brightness variation.

This fault is central to the project because it directly tests whether the student understands the difference between fixed reference terminals and the variable wiper terminal. The outer terminals establish the voltage range. The wiper terminal produces the changing signal. The correct connection of A0 to the center terminal is therefore not optional; it is the condition that makes analog control possible.

5.4.1 Fault Description

The fault occurs when Arduino analog input A0 is connected to one of the two outer terminals of the potentiometer instead of the center wiper terminal. In this faulty arrangement, the potentiometer may still be connected between 5V and GND, and the LED circuit may still be connected correctly to D9 through the 220Ω resistor. However, the Arduino input will not receive the variable voltage produced by the wiper.

This fault can be visually identified in the schematic by examining the wire connected to A0. If the A0 wire terminates at the top or bottom terminal of the potentiometer, the circuit is incorrect. The A0 wire must terminate at the center terminal, which is the moving contact of the potentiometer.

The fault is serious because the Arduino program depends entirely on the value read at A0. If A0 receives a fixed voltage, the program will still run, but it will process a constant input value. The resulting PWM output will also remain fixed or nearly fixed. Therefore, the system may appear powered and active, but it will fail to perform its intended analog control function.

5.4.2 Electrical Reason for the Fault

The electrical reason for this fault is that the outer terminals of a potentiometer do not provide the variable voltage. The outer terminals are the endpoints of the resistive track. When one endpoint is connected to 5V and the other endpoint is connected to GND, these terminals remain fixed at their respective reference potentials. The center wiper is the only terminal that changes its voltage as the knob is rotated.

If A0 is connected to the 5V side, then the voltage at A0 is approximately:

$$V_A0 \approx 5V$$

The expected ADC value becomes:

$$ADC = (5V \div 5V) \times 1023 = 1023$$

The expected PWM value becomes:

$$PWM = 1023 \div 4 \approx 255$$

The LED therefore remains at maximum brightness.

If A0 is connected to the GND side, then the voltage at A0 is approximately:

$$V_A0 \approx 0V$$

The expected ADC value becomes:

$$ADC = (0V \div 5V) \times 1023 = 0$$

The expected PWM value becomes:

$$PWM = 0 \div 4 = 0$$

The LED therefore remains off.

In both cases, the control system fails because the Arduino receives a fixed endpoint voltage instead of the variable wiper voltage. The mathematical consequence is direct: a constant input voltage produces a constant ADC value, and a constant ADC value produces a constant PWM value. Therefore, the LED brightness cannot vary.

5.4.3 Observed Result

The observed result of this fault is that the LED brightness does not respond correctly to potentiometer rotation. If A0 is connected to the outer terminal tied to 5V, the LED may remain fully bright. If A0 is connected to the outer terminal tied to GND, the LED may remain off. In some cases, depending on the wiring and simulation model, the LED may remain at a fixed brightness level.

The key observation is not merely that the LED is bright or dark. The key observation is that the LED does not change when the knob is rotated. This means that the input value being processed by the Arduino is not changing. The most direct diagnostic test is to measure the voltage between A0 and GND while rotating the potentiometer. If the voltage remains constant, then A0 is not receiving the wiper signal.

A correct system should show a changing voltage at A0. A faulty system with A0 connected to an outer pin will show a fixed voltage. Therefore, the observed result is consistent with the electrical cause.

5.4.4 Corrective Action

The corrective action is to move the A0 connection from the outer potentiometer terminal to the center wiper terminal. The two outer terminals must remain connected to 5V and GND. The center terminal must be connected to Arduino A0. After this correction, rotating the potentiometer should cause the voltage at A0 to vary between approximately 0V and 5V.

After the wire has been moved, the circuit should be verified in three steps. First, the voltage at A0 should be measured while the potentiometer is rotated. Second, the ADC value should be observed through code or simulation. Third, the LED brightness should be checked visually. If these three observations agree, the fault has been corrected.

The corrected relationship should be:

$$0V \leq V_A0 \leq 5V$$

$$0 \leq ADC \leq 1023$$

$$0 \leq PWM \leq 255$$

When these three ranges are satisfied and the LED brightness changes accordingly, the potentiometer connection is correct.

5.5 A0 Connected to an Outer Potentiometer Pin

When A0 is connected to an outer potentiometer pin, the Arduino reads one of the voltage endpoints rather than the changing wiper voltage. This is one of the most common and most important errors in this project. The error is caused by misunderstanding the function of the three potentiometer terminals.

If A0 is connected to the 5V outer terminal, the analog input remains near 5V. The Arduino reading remains near 1023. The mapped PWM value remains near 255. As a result, the LED remains fully bright or nearly fully bright.

If A0 is connected to the GND outer terminal, the analog input remains near 0V. The Arduino reading remains near 0. The mapped PWM value remains near 0. As a result, the LED remains off or nearly off.

The solution is to identify the center wiper terminal and connect it to A0. The outer terminals may be connected to 5V and GND in either order. Swapping the outer terminals changes the direction of control, meaning clockwise rotation may increase brightness in one arrangement and decrease brightness in the other. However, the center wiper must always be connected to A0.

5.6 Potentiometer Wiper Not Connected

If the potentiometer wiper is not connected to Arduino A0, the analog input may become floating. A floating input is an input that is not tied to a stable voltage reference. In this condition, the Arduino may read unpredictable values because the pin can be affected by electrical noise, nearby signals, and simulation instability.

The observed result may be unstable LED brightness, random flickering, irregular changes, or no meaningful response to knob rotation. The values read by A0 may jump suddenly rather than change smoothly. This behavior is different from the outer pin fault. In the outer pin fault, the reading is usually fixed. In the disconnected wiper fault, the reading may be unstable.

The correction is to connect the potentiometer center wiper directly to A0. The connection should be short, clear, and electrically continuous. After correction, the A0 voltage should vary smoothly with the potentiometer position.

A proper diagnostic check is to measure the voltage between A0 and GND. If the wiper is disconnected, the voltage may be undefined or unstable. If the wiper is correctly connected, the voltage should remain within the controlled range from 0V to 5V and should change according to the knob position.

5.7 LED Does Not Turn On

If the LED does not turn on, the fault may exist in the wiring, the component selection, the Arduino code, or the Proteus simulation setup. This symptom must be examined carefully because several different faults can produce the same visible result.

The LED remaining off does not automatically mean that the LED is damaged. It may mean that the LED is reversed, that D9 is not producing a PWM signal, that the resistor is not connected correctly, that there is no ground return, that the Arduino

program has not been loaded, or that the potentiometer input is forcing the PWM value to zero.

The first diagnostic step is to confirm that the potentiometer is not at the minimum position. If the potentiometer is correctly at minimum, the LED being off is normal. The second diagnostic step is to increase the potentiometer position and observe whether the LED responds. If the LED remains off at high potentiometer position, then a fault exists.

The expected condition at maximum position is:

$$V_A 0 \approx 5V$$

$$ADC \approx 1023$$

$$PWM \approx 255$$

If these values are present and the LED remains off, the fault is likely in the output stage. If these values are not present, the fault is likely in the input stage or code.

5.7.1 Possible Wiring Causes

The LED may fail to turn on because the output wiring is incomplete or incorrect. The most common wiring causes include a missing connection between D9 and the resistor, a missing connection between the resistor and the LED anode, a missing connection between the LED cathode and GND, or a disconnected common ground.

Another wiring cause is placing the resistor incorrectly so that it is not in series with the LED. The resistor must be part of the current path from D9 to the LED and then to GND. If the resistor is placed in a disconnected branch, it will not protect the LED and may not allow current to flow through the intended path.

A reversed LED connection is also a wiring fault. The LED anode must face the positive drive side, and the cathode must return to GND. If the LED is reversed, it blocks current in the intended direction and remains off.

The wiring path must therefore be:

$$D9 \rightarrow 220\Omega resistor \rightarrow LEDanode \rightarrow LEDcathode \rightarrow GND$$

If any part of this path is broken, the LED may remain off.

5.7.2 Possible Component Causes

The LED may fail to turn on because of a component related problem. The LED component may be damaged, incorrectly selected, or incorrectly oriented. In Proteus, the selected LED model must be appropriate for visible output. A wrong component model may not display the expected behavior.

The resistor value may also be incorrect. If the resistor value is extremely high, the current may be too small to produce visible brightness. For example, a resistor of

100kΩ would severely limit current, and the LED may appear off. The intended resistor value in this project is 220Ω.

The potentiometer may also be incorrectly valued or incorrectly configured. A very high resistance value may make the analog input more sensitive to noise, while an incorrectly connected potentiometer may prevent the intended voltage range from appearing at A0. The selected value of 10k is appropriate because it provides a practical balance between stable voltage division and low current consumption.

5.7.3 Possible Code Causes

The LED may remain off because the Arduino code does not write a valid PWM value to the output pin. The output pin must be declared correctly, and the program must use the correct pin number. In this project, the output pin is D9.

The essential program logic is:

int sensorValue = analogRead(A0);

int pwmValue = map(sensorValue, 0, 1023, 0, 255);

analogWrite(9, pwmValue);

If the program writes to a different pin, the LED connected to D9 will not respond. If the program always writes zero, the LED will remain off. If the analog value is not read from A0, the PWM output may not represent the potentiometer position.

A code fault can be separated from a wiring fault by testing a fixed PWM command. If analogWrite(9, 255) turns the LED on, then the LED branch is functional and the problem is likely related to analog reading or mapping. If analogWrite(9, 255) does not turn the LED on, the problem is likely in the output wiring, LED orientation, resistor path, or simulation setup.

5.7.4 Possible Simulation Causes

In Proteus, the LED may remain off because the Arduino simulation is not correctly configured. The most common simulation cause is that the compiled HEX file has not been loaded into the Arduino UNO component. Without the program file, the Arduino model cannot execute the intended code.

Another simulation cause is the absence of a ground terminal. Proteus circuits require a defined ground reference. If the circuit lacks GND, simulation behavior may fail or become undefined. The Arduino, potentiometer, and LED branch must share the same ground reference.

A third simulation cause is selecting an incorrect component model. If the Arduino model, potentiometer model, LED model, or resistor configuration is unsuitable, the circuit may not behave as expected. Simulation settings, clock frequency, and program loading should also be checked.

The simulation should be restarted after major wiring corrections. This ensures that the updated circuit and loaded program are both active during testing.

5.8 LED Always Fully Bright

If the LED is always fully bright, the output pin is likely receiving or producing a maximum or near maximum PWM value. This condition can occur for several reasons.

The most common cause is that A0 is connected to the 5V outer terminal of the potentiometer. In this case:

$$V_A0 \approx 5V$$

$$ADC \approx 1023$$

$$PWM \approx 255$$

The LED remains fully bright because the Arduino believes that the potentiometer is always at maximum position.

Another possible cause is that the code writes a constant value of 255 to D9. This may happen if the code was written for simple LED testing and was not updated for analog control. The program must read A0 and map the result to a PWM value.

A further possible cause is an accidental connection between 5V and the LED branch. If the LED is driven directly from 5V rather than from D9 through the program controlled PWM path, the Arduino will not control brightness.

The correction process is as follows. First, check whether A0 is connected to the wiper. Second, measure the voltage at A0 while rotating the potentiometer. Third, confirm that the code writes the mapped PWM value to D9. Fourth, confirm that the LED branch is connected to D9 rather than directly to 5V.

5.9 LED Always Off

If the LED is always off, the circuit may be receiving a zero input value, failing to produce PWM output, or failing to complete the LED current path. This symptom must be distinguished from the normal condition in which the LED is off only when the potentiometer is at the minimum position.

One common cause is that A0 is connected to the GND outer terminal of the potentiometer. In this case:

$$V_A0 \approx 0V$$

$$ADC \approx 0$$

$$PWM \approx 0$$

The LED remains off because the Arduino continuously produces a zero PWM output.

Another cause is reversed LED polarity. If the cathode and anode are reversed,

the LED blocks current and does not emit light. A third cause is a missing ground connection. Without a complete return path, current cannot flow through the LED.

A code related cause is writing to the wrong pin. If the code writes PWM to pin 9 but the LED is connected to another pin, or if the LED is connected to pin 9 but the code writes to another pin, the LED will not respond.

The correction process is to verify the A0 voltage range, verify the D9 PWM output, verify the LED polarity, verify the resistor path, and verify the ground return.

5.10 LED Brightness Does Not Change

If the LED turns on but its brightness does not change, the circuit may have a fixed input, a fixed output, or an incorrect mapping process. This symptom is especially important because it indicates that the circuit is partially working but the analog control function is not working.

The most likely input fault is that A0 is connected to an outer potentiometer terminal rather than to the wiper. In that case, the LED may remain off, fully bright, or at a fixed brightness depending on which fixed voltage is being read.

The most likely code fault is that the program writes a constant PWM value instead of using the mapped analog reading. For example, if the program contains analogWrite(9, 128), the LED will remain at a medium brightness regardless of potentiometer position.

The most likely simulation fault is that the potentiometer model is not being adjusted during simulation, or the Arduino program file has not been updated after code changes.

The correct diagnosis requires observing the A0 voltage. If the A0 voltage changes but the LED brightness does not change, the problem is likely after the analog input stage. If the A0 voltage does not change, the problem is likely in the potentiometer wiring.

5.11 LED Flickering or Unstable Brightness

LED flickering or unstable brightness can occur when the analog input is unstable, the wiper connection is loose, the ground reference is poor, or the simulation values are fluctuating. Unlike the fixed brightness fault, this condition usually indicates that the input or output is changing unpredictably.

A floating A0 input is one of the most common causes. If the wiper is not firmly connected to A0, the analog input can read random values. These values are then mapped to random PWM values, causing the LED brightness to flicker.

Another possible cause is a weak or missing ground reference. Since the analog reading is measured relative to GND, an unstable ground affects the measured voltage. The potentiometer ground and LED ground must return to the Arduino ground.

A software improvement can reduce minor fluctuations by averaging multiple analog readings. A simple averaging method is:

$$average = sum of several analog readings \div number of readings$$

For ten readings, the concept is:

$$averageADC = (ADC1 + ADC2 + ADC3 + ADC4 + ADC5 + ADC6 + ADC7 + ADC8 + ADC9 + ADC10) \div 10$$

The averaged value can then be mapped to PWM. This reduces small variations and produces smoother LED brightness. However, smoothing should not be used to hide a wiring fault. The wiring must first be correct.

5.12 Wrong LED Polarity

A red LED is a polarized component. This means it conducts current mainly in one direction. If the LED is connected with the wrong polarity, it will not turn on under normal operating conditions.

In the correct connection, the resistor is connected to the LED anode, and the LED cathode is connected to GND. The current path is:

$$D9 \rightarrow 220\Omega resistor \rightarrow LEDanode \rightarrow LEDcathode \rightarrow GND$$

If the LED is reversed, the current path is blocked. The circuit may appear correct in terms of wire placement, but the LED will remain off.

The correction is to reverse the LED orientation. In a physical circuit, the longer LED leg is usually the anode and the shorter leg is usually the cathode. In Proteus, the schematic symbol must be oriented so that current flows from the Arduino output side through the LED toward GND.

Wrong LED polarity is a simple fault, but it is academically important because it shows the difference between a non polarized component, such as a resistor, and a polarized component, such as an LED.

5.13 Missing Current Limiting Resistor

The current limiting resistor is essential for safe LED operation. If the LED is connected directly between D9 and GND without a resistor, excessive current may flow. In a real physical circuit, this can damage the LED and may also stress the Arduino output pin.

The resistor limits current according to Ohm's law. Assuming a 5V output and a red LED forward voltage of approximately 2V, the voltage across the resistor is:

$$V_R = 5V - 2V = 3V$$

With a 220Ω resistor, the approximate current is:

$$I = V_R \div R$$

$$I = 3V \div 220\Omega$$

$$I \approx 0.0136A$$

$$I \approx 13.6mA$$

This current is appropriate for an educational LED circuit. Without the resistor, the current would not be properly limited by the circuit design. The LED and Arduino pin would depend on internal resistance and device limits, which is unsafe and unacceptable for a correct report.

The correction is to place the 220Ω resistor in series with the LED. It may be placed before the LED or after the LED, as long as it remains in the same current path. In this project, the recommended path is D9 to resistor to LED anode to LED cathode to GND.

5.14 Missing Common Ground

A missing common ground is a serious fault because the circuit requires a shared voltage reference. The Arduino, potentiometer, LED branch, and simulation instruments must refer to the same GND. Without this shared reference, voltage values may become undefined, and the circuit may not behave as expected.

The analog input A0 measures voltage relative to Arduino GND. Therefore, the potentiometer ground must be connected to Arduino GND. If the potentiometer is connected to 5V but its lower terminal is not connected to Arduino GND, the voltage divider is incomplete.

The LED output branch also requires a return path to GND. If the LED cathode is not connected to GND, current cannot return to the source, and the LED may remain off.

In Proteus, a ground terminal is often required for simulation stability. Even if wires appear connected, the circuit may not simulate correctly without an explicit ground reference.

The correction is to connect all ground points to the same electrical ground node. The potentiometer GND, LED cathode return, Arduino GND, and simulation ground reference must all be common.

5.15 Wrong Arduino Pin Selection

Wrong Arduino pin selection can prevent the circuit from working even when the wiring appears mostly correct. In this project, A0 must be used for analog input, and D9 must be used for PWM output. If another pin is selected accidentally, the code and schematic will not match.

A common mistake is connecting the LED to D9 while the code writes PWM to another digital pin. Another common mistake is connecting the potentiometer to A0 while the code reads from a different analog input. In both cases, the physical circuit and the program logic are inconsistent.

The correct pin assignment is:

$$Potentiometerwiper \rightarrow A0$$

$$PWMoutput \rightarrow D9$$

The code must match these assignments exactly:

const int potPin = A0;

const int ledPin = 9;

If the hardware and code use different pins, the result may be no LED response, constant brightness, or incorrect behavior. The correction is to make the schematic and the program consistent.

5.16 HEX File Not Loaded in Proteus

In Proteus simulation, the Arduino UNO model does not automatically execute the source code written in the Arduino IDE. The program must first be compiled, and the resulting HEX file must be loaded into the Arduino component properties in Proteus. If the HEX file is missing, outdated, or incorrectly loaded, the simulation will not represent the intended program.

This fault can produce several symptoms. The LED may remain off. The LED may not respond to the potentiometer. The Arduino may appear present in the schematic but functionally inactive. The circuit wiring may be correct, but the simulated controller will not perform the required logic.

The correction is to compile the Arduino program, locate the generated HEX file, open the Arduino UNO properties in Proteus, and assign the correct HEX file to the program file field. After loading the file, the simulation should be restarted.

The loaded program must correspond to the current version of the code. If the code has been changed but the HEX file has not been regenerated and reloaded, the simulation may still run the old behavior.

5.17 Troubleshooting Table

The troubleshooting table organizes faults, symptoms, causes, and corrections in a compact academic format. It allows the reader to compare different faults and identify the most likely solution based on the observed behavior.

5.18 Fault to Solution Matrix

The fault to solution matrix provides a more diagnostic method than the general troubleshooting table. It begins with the observed symptom and guides the investigation toward the most likely fault.

This matrix is important because it prevents uncontrolled guessing. Each symptom directs attention to a limited number of probable causes, and each cause has a specific correction.

Table 5.1: *Troubleshooting table*

Fault	Observed Symptom	Technical Cause	Correction
A0 connected to 5V outer pin	LED always fully bright	A0 reads approximately 5V	Move A0 to the center wiper
A0 connected to GND outer pin	LED always off	A0 reads approximately 0V	Move A0 to the center wiper
Wiper not connected	LED flickers or behaves randomly	A0 is floating	Connect wiper directly to A0
LED reversed	LED does not turn on	LED blocks current direction	Reverse LED polarity
Resistor missing	Unsafe LED branch	Current is not properly limited	Add 220Ω resistor in series
Missing common ground	Circuit unstable or inactive	No shared voltage reference	Connect all GND points together
Wrong output pin	LED does not respond	Code and wiring use different pins	Match code pin and circuit pin
HEX file not loaded	Simulation inactive	Arduino has no executable program	Load correct HEX file into Proteus
Constant PWM in code	Brightness does not change	Program ignores analog input	Use mapped A0 reading
Potentiometer not adjusted in simulation	No visible change	Input condition remains fixed	Rotate the simulated potentiometer

Table 5.2: *Fault to solution matrix*

Observed Condition	First Check	Second Check	Most Likely Correction
LED always bright	Measure A0 voltage	Check if A0 is tied to 5V	Connect A0 to wiper
LED always off	Measure A0 voltage	Check LED polarity and D9 output	Correct A0, LED polarity, or D9 path
LED does not change	Rotate potentiometer and measure A0	Check program mapping	Connect wiper to A0 or correct code
LED flickers	Check wiper continuity	Check common ground	Secure wiper and ground connections
Simulation does not run	Check Arduino program file	Check ground terminal	Load HEX file and add GND
PWM absent at D9	Check code pin number	Check compiled file	Use pin 9 in code and reload program
A0 value fixed at 1023	Check A0 wire	Check potentiometer endpoint	Move A0 from 5V side to wiper
A0 value fixed at 0	Check A0 wire	Check potentiometer endpoint	Move A0 from GND side to wiper
LED branch overheats in real hardware	Check resistor	Check direct LED connection	Add correct current limiting resistor

5.19 Figure Placement for Troubleshooting Chapter

Figures in the troubleshooting chapter must be placed where they support the explanation directly. A troubleshooting figure should not be decorative. It must clarify the fault, show the corrected connection, or demonstrate the electrical reason behind the observed behavior.

The chapter should contain at least four figures. The first figure should show the wrong potentiometer wiring. The second figure should show the corrected potentiometer wiring. The third figure should show the LED polarity fault. The fourth figure should show the missing ground fault.

Each figure must include a clear caption. The caption must state what the figure shows, why the shown condition is incorrect or correct, and what result is expected from that configuration.

5.19.1 Wrong Potentiometer Wiring Figure

The wrong potentiometer wiring figure should show A0 connected to an outer terminal of the potentiometer. The figure must clearly identify the three potentiometer pins and must mark the center wiper as unused or incorrectly bypassed. The purpose of this figure is to show why the analog input does not vary when the knob is rotated.

The figure caption may be written as follows:

Figure 5.1. Incorrect potentiometer wiring in which Arduino A0 is connected to an outer terminal instead of the center wiper. This connection causes A0 to read a fixed endpoint voltage rather than a variable control voltage.

The figure should include the following labels: 5V, GND, A0, wiper, outer terminal, and fixed voltage. If A0 is connected to the 5V side, the figure should indicate that the expected ADC value is close to 1023. If A0 is connected to the GND side, the figure should indicate that the expected ADC value is close to 0.

5.19.2 Corrected Potentiometer Wiring Figure

The corrected potentiometer wiring figure should show one outer terminal connected to 5V, the other outer terminal connected to GND, and the center wiper connected to A0. This figure provides the correct reference against which the faulty figure is compared.

The figure caption may be written as follows:

Figure 5.2. Correct potentiometer wiring in which the center wiper terminal is connected to Arduino A0. This configuration allows the analog input voltage to vary from approximately 0V to 5V as the knob is rotated.

The figure should show the correct voltage relationship:

$$0V \leq V_A0 \leq 5V$$

It should also show that the ADC reading changes across the expected range:

$$0 \leq ADC \leq 1023$$

This figure is essential because it turns the troubleshooting explanation into a visually verifiable correction.

5.19.3 LED Polarity Fault Figure

The LED polarity fault figure should show the LED connected in reverse direction. The purpose of this figure is to explain why the LED remains off even when the Arduino output pin and resistor are connected.

The figure caption may be written as follows:

Figure 5.3. Incorrect LED polarity in the output branch. When the LED is reversed, current does not flow through the LED in the intended direction, and the LED remains off.

The figure should identify the anode, cathode, resistor, D9 pin, and GND return. It should also show the correct current path for comparison:

$$D9 \rightarrow 220\Omega resistor \rightarrow LEDanode \rightarrow LEDcathode \rightarrow GND$$

This figure helps separate LED orientation faults from code faults and potentiometer faults.

5.19.4 Missing Ground Fault Figure

The missing ground fault figure should show a circuit in which one or more ground points are not connected to the Arduino ground. This may include a disconnected potentiometer ground, a disconnected LED cathode return, or the absence of a ground terminal in Proteus.

The figure caption may be written as follows:

Figure 5.4. Missing common ground condition in the analog control circuit. Without a shared ground reference, the analog input and LED output branch cannot operate reliably.

The figure should show that the potentiometer, Arduino, and LED branch must share the same ground node. It should also identify the possible result of missing ground, such as unstable analog readings, inactive LED output, or failed simulation behavior.

5.20 Final Troubleshooting Checklist

The final troubleshooting checklist provides a complete verification sequence before the project is considered correct. Each item should be checked in order.

Confirm that the Arduino UNO R3 is present in the Proteus schematic.

Confirm that the potentiometer value is 10k.

Confirm that one outer potentiometer terminal is connected to 5V.

Confirm that the other outer potentiometer terminal is connected to GND.

Confirm that the center wiper terminal is connected to A0.

Confirm that A0 is not connected to an outer potentiometer terminal.

Confirm that the voltage at A0 changes when the potentiometer is rotated.

Confirm that the A0 voltage remains between 0V and 5V.

Confirm that D9 is used as the PWM output pin.
Confirm that the Arduino code reads from A0.
Confirm that the Arduino code writes PWM to pin 9.
Confirm that the ADC value is mapped from 0 to 1023 into 0 to 255.
Confirm that D9 is connected to the 220Ω resistor.
Confirm that the resistor is connected in series with the LED.
Confirm that the LED anode is connected to the resistor side.
Confirm that the LED cathode is connected to GND.
Confirm that all GND points share the same reference.
Confirm that the LED is not connected directly without a resistor.
Confirm that the Proteus ground terminal is present.
Confirm that the correct HEX file is loaded into the Arduino component.
Confirm that the simulation is restarted after loading the HEX file.
Confirm that the potentiometer is adjusted during simulation testing.
Confirm that low potentiometer position produces low brightness.
Confirm that middle potentiometer position produces medium brightness.
Confirm that high potentiometer position produces high brightness.
Confirm that the LED brightness changes smoothly rather than randomly.
Confirm that no short circuit exists between 5V and GND.
Confirm that no required wire is left floating.
Confirm that the final circuit matches the scheme chapter.
Confirm that the observed result matches the calculation and simulation chapters.

5.21 Chapter Summary

This chapter examined the major faults that may occur in the Arduino potentiometer based LED brightness control project. The analysis showed that the most important fault is the incorrect connection of Arduino A0 to an outer potentiometer terminal instead of the center wiper. This error prevents the Arduino from receiving a variable voltage and causes the LED brightness to remain fixed.

The chapter also explained several other possible faults, including a disconnected wiper, reversed LED polarity, missing current limiting resistor, missing common ground, wrong Arduino pin selection, and missing HEX file in Proteus. Each fault was connected to its technical cause, visible symptom, and required correction.

The correct operation of the circuit depends on a complete and consistent chain of behavior. The potentiometer must generate a variable voltage at A0. The Arduino must convert this voltage into an ADC value. The program must map the ADC value into a PWM output. The D9 pin must drive the LED through a 220Ω resistor. The LED must be correctly oriented and connected to ground. The Proteus simulation must contain the correct program file and a valid ground reference.

The troubleshooting process therefore confirms that the project is not only functionally successful but also technically understood. A working LED is not sufficient evidence by itself. The system is verified only when the input voltage, ADC value, PWM output, LED brightness, wiring structure, and simulation behavior all agree with the theoretical design.

6 Questions

6.1 Introduction to Evaluation Questions

This chapter presents the evaluation questions designed to measure the learner's understanding of the analog control system developed in this project. The questions are organized according to the main knowledge areas of the project, beginning with basic concepts and progressing toward calculation, circuit interpretation, simulation analysis, troubleshooting, and possible system extensions. The structure of this chapter ensures that the evaluation does not only test memory, but also tests reasoning, accuracy, electrical understanding, and the ability to explain the relationship between hardware, software, and simulation behavior.

The purpose of these evaluation questions is to confirm that the learner can explain how a potentiometer produces a variable voltage, how the Arduino UNO reads that voltage through an analog input, how the analog reading is converted into a pulse width modulation value, and how the pulse width modulation signal controls the brightness of an LED through a current limiting resistor. The questions also verify whether the learner can identify incorrect wiring conditions, interpret Proteus simulation results, perform basic electrical calculations, and suggest suitable improvements to the circuit.

This chapter is placed at the end of the report because it depends on all previous chapters. The description chapter provides the conceptual foundation, the calculation chapter provides the mathematical foundation, the scheme chapter provides the wiring foundation, the simulation chapter provides the behavioral foundation, and the troubleshooting chapter provides the diagnostic foundation. Therefore, Chapter 6 functions as the final academic verification stage of the complete project.

6.2 Basic Concept Questions

This section evaluates the learner's general understanding of the project. The questions in this section focus on the meaning of analog control, the role of the potentiometer, the purpose of the Arduino UNO, and the reason the LED brightness changes when the knob is rotated. These questions are necessary because a learner must first understand

the conceptual operation of the system before attempting mathematical calculation or circuit diagnosis.

The main concept of the project is that a physical action, namely turning the potentiometer knob, is converted into an electrical signal. This electrical signal is then interpreted by the Arduino and used to control the LED. The learner must understand that the potentiometer does not directly control the LED brightness in this circuit. Instead, the potentiometer produces a variable voltage, the Arduino reads that voltage, and the Arduino produces a pulse width modulation output that controls the LED.

Suitable questions for this section include the following.

What is the main purpose of the analog control system in this project?

What physical action changes the input signal in this circuit?

Why is the potentiometer used as an input device?

What does the Arduino UNO do after reading the potentiometer value?

Why does the LED brightness change when the potentiometer is rotated?

What is the difference between an analog input signal and a digital output signal?

Why is this project considered an example of analog control?

What is meant by converting physical movement into an electrical signal?

Why is the center terminal of the potentiometer important?

What is the complete signal path from the potentiometer to the LED?

The expected understanding is that the learner can describe the system as a chain of conversion. Mechanical rotation changes the voltage at the potentiometer wiper. The Arduino reads this voltage at analog pin A0. The Arduino converts the analog reading into a numerical value. The numerical value is mapped to a pulse width modulation value. The pulse width modulation signal on digital pin D9 controls the average power delivered to the LED. The LED brightness changes according to the duty cycle of the pulse width modulation signal.

6.3 Component Identification Questions

This section evaluates whether the learner can correctly identify every component used in the project and explain its function. Component identification is important because a circuit cannot be understood only from its final behavior. The learner must be able to identify the role of each physical part and explain why it is included in the system.

The project uses an Arduino UNO R3, a 10 kΩ potentiometer, a 220 Ω resistor, a red LED, jumper wires, power connections, and ground connections. In Proteus, these components must be selected correctly and connected according to the circuit scheme. Each component has a separate role. The Arduino acts as the controller, the potentiometer acts as the analog input device, the resistor limits LED current, and the LED acts as the visible output device.

Suitable questions for this section include the following.

Which component is used as the main controller in the circuit?

Which component is used to produce the variable analog voltage?

Which component is used as the visible output device?

Which component protects the LED from excessive current?

What is the value of the potentiometer used in this project?

What is the value of the resistor used in series with the LED?

Which Arduino pin receives the potentiometer signal?

Which Arduino pin sends the pulse width modulation signal to the LED?

What is the function of the ground connection in this circuit?

Why must the Arduino, potentiometer, and LED branch share a common ground?

The learner should be able to answer that the Arduino UNO R3 is the controller, the 10 kΩ potentiometer is the input component, the red LED is the output component, and the 220 Ω resistor is the current limiting component. The potentiometer wiper is connected to A0. The LED is driven from D9 through the resistor. The ground connection provides the common reference point for all voltage measurements and current paths.

6.4 Potentiometer Questions

This section evaluates detailed understanding of the potentiometer. The potentiometer is the most important input component in the project, and misunderstanding its terminals is the most common cause of circuit failure. Therefore, this section must test both conceptual and wiring knowledge.

A potentiometer has three terminals. The two outer terminals are connected to the ends of the resistive track. In this project, one outer terminal is connected to 5 V and the other outer terminal is connected to ground. The center terminal is the wiper. The wiper moves along the resistive track when the knob is rotated. This movement changes the output voltage at the center terminal. The Arduino reads this variable voltage from analog pin A0.

Suitable questions for this section include the following.

How many terminals does a standard potentiometer have?

What is the function of the two outer terminals?

What is the function of the center terminal?

Why is the center terminal called the wiper?

Which potentiometer terminal must be connected to Arduino A0?

What happens if A0 is connected to an outer terminal instead of the wiper?

Why are the outer terminals connected to 5 V and ground?

What happens if the two outer terminals are swapped?

Why does the potentiometer act as a voltage divider in this circuit?

What voltage should appear at the wiper when the knob is near the middle position?

The learner should understand that the potentiometer is not being used as a simple variable resistor in this specific circuit. It is being used as a voltage divider. The wiper voltage varies between 0 V and 5 V. If the wiper is connected correctly to A0, the Arduino receives a changing input value. If A0 is connected to an outer pin, the Arduino receives a fixed voltage and the LED brightness does not change.

6.5 Arduino Analog Input Questions

This section evaluates the learner's understanding of Arduino analog input behavior. The Arduino UNO cannot directly understand continuous voltage as a human observer would. Instead, it samples the voltage and converts it into a digital number using its analog to digital converter. This section checks whether the learner understands the meaning of analog reading and the limitation of the analog input range.

In this project, analog pin A0 reads the voltage coming from the potentiometer wiper. Since the Arduino UNO normally uses a 5 V reference in this project, an input of 0 V corresponds to the lowest reading, and an input of 5 V corresponds to the highest reading. The analog input must never be connected to voltages outside the allowed range, because doing so may damage the microcontroller.

Suitable questions for this section include the following.

Which Arduino pin is used to read the potentiometer signal?

What type of signal is read by Arduino pin A0?

What is the minimum expected voltage at A0?

What is the maximum expected voltage at A0?

What function is used in Arduino code to read the analog input?

What does the value returned by analogRead represent?

Why does the Arduino need an analog to digital converter?

What happens when the voltage at A0 increases?

What happens when the voltage at A0 decreases?

Why must the potentiometer ground and Arduino ground be connected together?

The learner should answer that A0 reads the variable voltage from the potentiometer wiper. The expected voltage range is from 0 V to 5 V. The analogRead function converts the voltage into a numerical value. As the voltage increases, the ADC value increases. As the voltage decreases, the ADC value decreases. The common ground is necessary because voltage is always measured relative to a reference point.

6.6 ADC Calculation Questions

This section evaluates mathematical understanding of analog to digital conversion. The Arduino UNO analog input produces a 10 bit reading, which gives values from 0 to 1023. The learner must be able to calculate the expected ADC value for a given voltage and calculate the approximate voltage from a given ADC value.

The basic relationship is:

$$ADC = \frac{V_{A0}}{V_{REF}} \times 1023$$

In this project:

$$V_{REF} = 5V$$

Therefore:

$$ADC = \frac{V_{A0}}{5V} \times 1023$$

The reverse relationship is:

$$V_{A0} = \frac{ADC}{1023} \times 5V$$

Suitable questions for this section include the following.
What is the ADC value when the voltage at A0 is 0 V?
What is the ADC value when the voltage at A0 is 2.5 V?
What is the ADC value when the voltage at A0 is 5 V?
What is the approximate ADC value when the voltage at A0 is 1.25 V?
What is the approximate ADC value when the voltage at A0 is 3.75 V?
If the ADC value is 512, what is the approximate input voltage?
If the ADC value is 767, what is the approximate input voltage?
Why is the highest ADC value 1023 instead of 1000?
What does 10 bit resolution mean in this project?
Why does the ADC value change when the potentiometer is rotated?
Expected calculations include the following.
For 0 V:

$$ADC = \frac{0}{5} \times 1023 = 0$$

For 2.5 V:

$$ADC = \frac{2.5}{5} \times 1023 = 511.5$$

The approximate integer reading is 512.
For 5 V:

$$ADC = \frac{5}{5} \times 1023 = 1023$$

For 3.75 V:

$$ADC = \frac{3.75}{5} \times 1023 = 767.25$$

The approximate integer reading is 767.

This section confirms whether the learner can connect physical voltage to numerical representation.

6.7 PWM Questions

This section evaluates the learner's understanding of pulse width modulation. The Arduino UNO does not produce a true continuously variable analog voltage on pin D9. Instead, it produces a digital signal that switches rapidly between low and high states. The percentage of time the signal remains high is called the duty cycle. The LED appears dimmer or brighter because the average delivered power changes as the duty cycle changes.

The Arduino analogWrite function accepts values from 0 to 255 for standard pulse width modulation output. A value of 0 represents fully off. A value of 255 represents

fully on. A value near 128 represents approximately half duty cycle.

Suitable questions for this section include the following.

What does PWM stand for?

Which Arduino pin is used for PWM output in this project?

What function is used to write the PWM value in Arduino code?

What is the minimum PWM value?

What is the maximum PWM value?

What does a PWM value of 0 mean?

What does a PWM value of 255 mean?

What does a PWM value of approximately 128 mean?

Why does PWM control LED brightness?

Why is PWM not the same as a true analog voltage output?

The learner should understand that D9 is used as the PWM output pin. The Arduino code reads the input value from A0 and maps it to a PWM value. The relationship can be written as:

$$PWM = \frac{ADC}{1023} \times 255$$

A simplified approximation is:

$$PWM \approx \frac{ADC}{4}$$

If the ADC value is 512, then:

$$PWM = \frac{512}{1023} \times 255$$

$$PWM \approx 128$$

If the ADC value is 767, then:

$$PWM = \frac{767}{1023} \times 255$$

$$PWM \approx 191$$

This section confirms that the learner understands how the analog input reading becomes an output control signal.

6.8 LED and Resistor Questions

This section evaluates whether the learner understands the electrical protection required for the LED. The LED must not be connected directly to an Arduino output pin without a resistor, because excessive current may damage the LED or the Arduino pin. The resistor limits the current to a safe value.

The LED has polarity. The anode must be connected toward the positive driving side, and the cathode must be connected toward ground. In this circuit, D9 connects

to the 220 Ω resistor, the resistor connects to the LED anode, and the LED cathode connects to ground.

Suitable questions for this section include the following.

Why is a resistor connected in series with the LED?

What is the value of the resistor used in this project?

What is the function of the LED in this circuit?

Which side of the LED is connected toward the resistor?

Which side of the LED is connected to ground?

What may happen if the resistor is removed?

What may happen if the LED is reversed?

Why is 220 Ω a suitable resistor value for this project?

How is LED current calculated?

How does PWM affect the apparent brightness of the LED?

The current calculation can be written as:

$$I = \frac{V_{SUPPLY} - V_{LED}}{R}$$

Using the project values:

$$V_{SUPPLY} = 5V$$

$$V_{LED} \approx 2V$$

$$R = 220\Omega$$

Therefore:

$$I = \frac{5V - 2V}{220\Omega}$$

$$I = \frac{3V}{220\Omega}$$

$$I \approx 0.0136A$$

$$I \approx 13.6mA$$

This result is suitable for an educational LED circuit because the current is limited to a reasonable value.

6.9 Circuit Scheme Questions

This section evaluates whether the learner can read, explain, and verify the circuit scheme. A correct scheme is essential because even correct code and correct calculations cannot compensate for incorrect wiring. The learner must be able to identify every connection and explain why each connection is necessary.

The circuit scheme must clearly show the potentiometer connected as a voltage divider and the LED connected as a current limited output load. The most important connection is the wiper to A0 connection. The second most important connection is the series path from D9 through the resistor and LED to ground.

Suitable questions for this section include the following.

Which Arduino pin is connected to the potentiometer wiper?

Which Arduino pin is connected to the LED branch?

Which potentiometer terminal is connected to 5 V?

Which potentiometer terminal is connected to ground?

Which potentiometer terminal is connected to A0?

Where is the 220 Ω resistor placed in the circuit?

Why must the resistor be placed in series with the LED?

Where is the LED cathode connected?

What does a common ground mean in the scheme?

How can the scheme be checked before running the simulation?

The learner should be able to provide the exact wiring table.

Arduino 5 V is connected to one outer pin of the potentiometer.

Arduino ground is connected to the other outer pin of the potentiometer.

The potentiometer center wiper is connected to Arduino A0.

Arduino D9 is connected to the 220 Ω resistor.

The 220 Ω resistor is connected to the LED anode.

The LED cathode is connected to Arduino ground.

This section proves that the learner can translate a visual schematic into precise electrical connections.

6.10 Proteus Simulation Questions

This section evaluates the learner's ability to understand and interpret simulation behavior in Proteus. Simulation is not only used to display a circuit. It is used to verify whether the theoretical design behaves correctly before physical implementation. The learner must be able to explain what should happen during simulation and how to recognize correct or incorrect operation.

In the Proteus simulation, the potentiometer position should change the voltage at A0. As the voltage at A0 changes, the ADC value changes. As the ADC value changes, the PWM value on D9 changes. As the PWM value changes, the LED brightness changes. The simulation should confirm this complete chain.

Suitable questions for this section include the following.

Why is Proteus simulation used in this project?

What file must be loaded into the Arduino model before simulation?

What should happen when the potentiometer is rotated in simulation?

What should the LED do when the potentiometer voltage increases?

What should the LED do when the potentiometer voltage decreases?

What should a voltmeter show when connected between A0 and ground?

What should an oscilloscope show when connected to D9?

What result indicates that the circuit is working correctly?

What result indicates that the potentiometer is incorrectly connected?

Why should simulation results be compared with calculated values?

The expected simulation behavior can be organized as follows.

At 0 percent potentiometer position, the A0 voltage should be close to 0 V, the ADC value should be close to 0, the PWM value should be close to 0, and the LED should be off.

At 50 percent potentiometer position, the A0 voltage should be close to 2.5 V, the ADC value should be close to 512, the PWM value should be close to 128, and the LED should show medium brightness.

At 100 percent potentiometer position, the A0 voltage should be close to 5 V, the ADC value should be close to 1023, the PWM value should be close to 255, and the LED should be fully bright.

This section confirms the learner's ability to connect simulation evidence to theoretical expectation.

6.11 Troubleshooting Questions

This section evaluates the learner's ability to diagnose faults. Troubleshooting questions are necessary because a learner may understand the correct circuit but still fail to identify why the circuit does not work. This section tests whether the learner can connect symptoms to causes and causes to corrections.

The main troubleshooting case in this project is connecting A0 to an outer potentiometer pin instead of the center wiper. In that case, the analog input receives a fixed voltage rather than a variable voltage. As a result, the LED brightness remains fixed when the knob is rotated.

Suitable questions for this section include the following.

What happens if A0 is connected to an outer potentiometer pin?

Why does the LED brightness remain unchanged in the wrong potentiometer connection?

What should be checked first if the LED does not turn on?

What should be checked first if the LED is always fully bright?

What should be checked first if the LED is always off?

What happens if the LED polarity is reversed?

What happens if the common ground connection is missing?

What happens if the 220 Ω resistor is removed?

What happens if the Arduino HEX file is not loaded in Proteus?

How can the correct wiper connection be verified?

The learner should understand the fault logic clearly.

If the LED is always bright, A0 may be fixed at 5 V, the code may be writing a constant maximum value, or the LED branch may be incorrectly connected to a constant high voltage.

If the LED is always off, A0 may be fixed at ground, the LED may be reversed, the resistor path may be open, D9 may not be producing output, or the simulation may not be running.

If the LED flickers unpredictably, A0 may be floating, the wiper connection may be unstable, or the simulation may contain an incomplete reference connection.

This section confirms diagnostic maturity rather than simple memorization.

6.12 Short Answer Questions

This section provides concise questions that require direct written answers. Short answer questions are useful because they test whether the learner can express essential ideas accurately without lengthy explanation. The answers should be brief, technically correct, and written using proper terminology.

Suitable short answer questions include the following.

Define analog input in the context of this project.

Define pulse width modulation.

State the purpose of the potentiometer.

State the purpose of the 220 Ω resistor.

State the function of Arduino pin A0.

State the function of Arduino pin D9.

State the meaning of ADC resolution.

State why the center potentiometer pin is important.

State why common ground is required.

State the expected LED behavior when the potentiometer is rotated.

Expected short answers should be precise. For instance, analog input is the variable voltage read by the Arduino from the potentiometer wiper. Pulse width modulation is a digital switching method that controls average output power by changing duty cycle. The 220 Ω resistor limits the current through the LED. The common ground provides a shared voltage reference for the circuit.

6.13 Calculation Based Questions

This section evaluates numerical competence. Calculation based questions require the learner to use formulas from the calculation chapter and apply them to specific values. This section must test voltage divider output, ADC conversion, PWM mapping, resistor selection, and LED current.

Suitable calculation based questions include the following.

If the potentiometer is at 25 percent position, calculate the approximate voltage at A0.

If the potentiometer is at 50 percent position, calculate the approximate voltage at A0.

If the potentiometer is at 75 percent position, calculate the approximate voltage at A0.

If A0 is 2.5 V, calculate the ADC value.

If A0 is 3.75 V, calculate the ADC value.

If the ADC value is 512, calculate the PWM value.

If the ADC value is 767, calculate the PWM value.

If the LED forward voltage is 2 V and the resistor is 220 Ω, calculate the LED current.

If the desired LED current is 15 mA, calculate the approximate resistor value.

If the supply voltage is 5 V and the LED forward voltage is 2 V, explain why 220 Ω is an acceptable resistor value.

The central formulas are:

$$V_{A0} = 5V \times p$$

$$ADC = \frac{V_{A0}}{5V} \times 1023$$

$$PWM = \frac{ADC}{1023} \times 255$$

$$R = \frac{V_{SUPPLY} - V_{LED}}{I}$$

$$I = \frac{V_{SUPPLY} - V_{LED}}{R}$$

Expected sample results include the following.

At 25 percent position:

$$V_{A0} = 5V \times 0.25 = 1.25V$$

At 50 percent position:

$$V_{A0} = 5V \times 0.50 = 2.5V$$

At 75 percent position:

$$V_{A0} = 5V \times 0.75 = 3.75V$$

For A0 equal to 2.5 V:

$$ADC = \frac{2.5V}{5V} \times 1023 \approx 512$$

For A0 equal to 3.75 V:

$$ADC = \frac{3.75V}{5V} \times 1023 \approx 767$$

For ADC equal to 512:

$$PWM = \frac{512}{1023} \times 255 \approx 128$$

For ADC equal to 767:

$$PWM = \frac{767}{1023} \times 255 \approx 191$$

For LED current:

$$I = \frac{5V - 2V}{220\Omega} \approx 13.6mA$$

This section ensures that the learner can convert conceptual knowledge into measurable numerical evidence.

6.14 Diagram Based Questions

This section evaluates the learner's ability to interpret circuit diagrams and Proteus schematic images. Diagram based questions are important because electrical understanding is not complete unless the learner can identify components, follow signal paths, and detect wiring faults from a visual representation.

The diagrams used in this section should include the correct circuit scheme, the calculation annotated scheme, the simulation running state, and the incorrect potentiometer wiring diagram. The learner should be asked to label connections, identify components, trace current paths, and locate faults.

Suitable diagram based questions include the following.

Identify the potentiometer in the circuit diagram.

Label the three terminals of the potentiometer.

Identify the wiper terminal in the diagram.

Trace the connection from the potentiometer wiper to the Arduino.

Identify the Arduino pin used for analog input.

Identify the Arduino pin used for PWM output.

Trace the current path from D9 to ground through the LED branch.

Identify the resistor in series with the LED.

Identify the LED anode and cathode.

Locate the wiring error in the incorrect potentiometer connection diagram.

Explain why the incorrect diagram causes fixed LED brightness.

Redraw or describe the correction needed to make the circuit operate properly.

The learner should demonstrate that the correct circuit has A0 connected to the center wiper, not to an outer terminal. The learner should also demonstrate that D9 must connect to the resistor first, then to the LED anode, and then from the LED cathode to ground. This section confirms visual and schematic literacy.

6.15 Practical Testing Questions

This section evaluates the learner's ability to test the circuit in practice. Practical testing questions are necessary because a working project must be verified through observation, measurement, and controlled changes. The learner must know what to measure, where to measure it, and how to interpret the results.

The practical tests should include potentiometer voltage measurement, LED brightness observation, PWM output verification,
continuity checking, polarity checking, and simulation comparison.
These tests allow the learner to confirm that each stage of the system is working.

Suitable practical testing questions include the following.

How can the voltage at A0 be measured?

Where should the voltmeter probes be placed to measure the potentiometer wiper voltage?

What should happen to the measured A0 voltage when the knob is rotated?

How can the PWM output on D9 be observed?

What should happen to the PWM duty cycle when the potentiometer voltage increases?

How can LED polarity be checked?

How can the resistor value be verified?

How can the common ground connection be tested?

How can the learner prove that the potentiometer is correctly connected?

How can the learner prove that the LED branch is correctly connected?

The correct measurement procedure is as follows. The positive voltmeter probe should be placed at the potentiometer wiper or at Arduino A0. The negative voltmeter probe should be placed at ground. When the potentiometer is rotated, the measured voltage should move smoothly from approximately 0 V to approximately 5 V. If the voltage does not change, the wiper connection is incorrect or disconnected.

The PWM signal can be observed with an oscilloscope or suitable simulation instrument. When the potentiometer voltage increases, the duty cycle should increase. When the potentiometer voltage decreases, the duty cycle should decrease. This practical testing section confirms that the learner can move from theory to verification.

6.16 Critical Thinking Questions

This section evaluates deeper reasoning. Critical thinking questions require the learner to explain causes, justify design choices, compare alternatives, and predict consequences. This section is important because it separates basic completion from genuine understanding.

The learner should not only know that the circuit works. The learner should be able to explain why the circuit works, why each component is necessary, and what would happen if the design were changed. This section should also require the learner to justify the use of simulation and explain the limitations of the project.

Suitable critical thinking questions include the following.

Why does the Arduino read voltage instead of resistance directly?

Why is the potentiometer configured as a voltage divider rather than connected as a two terminal variable resistor?

Why is PWM suitable for controlling LED brightness?

Why is a resistor required even when PWM is used?

Why does the LED appear dimmer when the duty cycle is reduced?

Why is Proteus simulation useful before physical construction?

What limitation exists when using an LED as the output device?

What limitation exists when using a potentiometer as the input device?

What would happen if the potentiometer value were extremely low?

What would happen if the potentiometer value were extremely high?

Why should a motor not be connected directly to Arduino D9?

How could the circuit be improved for smoother control?

Strong answers should show causal reasoning. The Arduino reads voltage because the analog input measures potential difference relative to ground. The potentiometer

is used as a voltage divider because this configuration produces a controlled variable voltage. PWM is suitable because the LED responds visually to average power over time. The resistor is still required because PWM does not remove the need for current limitation during the high part of the signal.

6.17 Extension Questions

This section evaluates whether the learner can transfer the knowledge gained from this project to related systems. Extension questions are important because the potentiometer based LED dimming project is a foundation for larger control systems. The same input and control principle can be applied to motors, displays, serial monitoring, sensors, and user interface systems.

The extension questions should encourage the learner to think beyond the basic circuit while preserving correct electrical principles. Any extension must respect current limits, voltage requirements, component protection, and signal compatibility.

Suitable general extension questions include the following.

How could this circuit be modified to control another output device?

How could the potentiometer reading be displayed to the user?

How could the circuit be modified to improve measurement stability?

How could serial communication be used to observe ADC and PWM values?

How could a sensor replace the potentiometer as the analog input?

How could the project be expanded into a closed loop control system?

What additional protection would be needed for higher power loads?

What software improvements could make the output smoother?

How could the circuit be adapted for classroom demonstration?

How could the project be expanded into a small embedded control module?

This section proves that the learner can apply the same principles to broader embedded system design.

6.17.1 Motor Speed Control Extension

This subsection evaluates how the project could be extended from LED brightness control to motor speed control. The learner must understand that a motor cannot be connected directly to an Arduino output pin in the same way as an LED. A motor usually requires more current than an Arduino pin can safely provide. Therefore, a transistor, MOSFET, or motor driver must be used.

The potentiometer can still act as the input device. The Arduino can still read the potentiometer voltage at A0. The Arduino can still generate PWM on a digital PWM pin. However, the PWM signal should control a driver stage, and the driver stage should control the motor power.

Suitable questions include the following.

Why should a DC motor not be connected directly to Arduino D9?

What additional component is required to control a motor safely?

How does PWM control motor speed?

Why is a flyback diode required when controlling a DC motor with a transistor?

How would the potentiometer still control the motor speed?

The expected answer is that the potentiometer provides the control input, the Arduino produces the PWM command, and the motor driver delivers the required motor current. This extension demonstrates the difference between signal control and power control.

6.17.2 LCD Display Extension

This subsection evaluates how the project could be extended by adding a display. An LCD could show the analog reading, the voltage value, the PWM value, or the brightness percentage. This extension improves observability and makes the system more informative for the user.

The learner should understand that the LCD does not change the basic control function. It adds a user information layer. The potentiometer remains the input, the Arduino remains the controller, the LED remains the output, and the LCD provides visual feedback.

Suitable questions include the following.

What values could be displayed on an LCD in this project?

How would displaying the ADC value help the learner understand the circuit?

How would displaying the PWM value help verify the output behavior?

What additional Arduino pins may be needed for an LCD?

Why is an LCD useful in an educational control project?

A strong answer should state that the LCD can display the ADC value from 0 to 1023, the calculated voltage from 0 V to 5 V, the PWM value from 0 to 255, and the brightness percentage from 0 percent to 100 percent.

6.17.3 Serial Monitor Extension

This subsection evaluates how the project could be extended by using serial communication. The Arduino Serial Monitor can display real time numerical values from the program. This is useful for debugging and for confirming whether the software values match the expected circuit behavior.

The learner should understand that the Serial Monitor can print the analog reading, calculated voltage, PWM output value, and brightness percentage. This makes the internal logic of the program visible without adding external display hardware.

Suitable questions include the following.

What is the purpose of the Serial Monitor in this project?

Which values should be printed to the Serial Monitor?

How can the Serial Monitor help diagnose a wrong potentiometer connection?

What serial command is used to begin serial communication in Arduino code?

Why is serial output useful during debugging?

A suitable serial output line could include the analog value, voltage, and PWM value. If the potentiometer is rotated but the ADC value remains unchanged, this indicates a wiring problem, a disconnected wiper, or an incorrect analog input connection.

6.17.4 Sensor Based Control Extension

This subsection evaluates how the potentiometer could be replaced by another analog input device. Many sensors provide variable voltage signals that can be read by the Arduino analog input. This extension shows that the project is not limited to manual control. It can become the foundation for sensor based automation.

A sensor based version could use a light dependent resistor circuit, temperature sensor, soil moisture sensor, pressure sensor, or any compatible analog voltage source. The Arduino would read the sensor value and control the LED or another output based on the measured condition.

Suitable questions include the following.

What type of sensor could replace the potentiometer?

How is a sensor based input similar to the potentiometer input?

How is a sensor based input different from a potentiometer input?

What must be checked before connecting a sensor to Arduino A0?

How could the LED brightness represent a sensor measurement?

A strong answer should explain that the potentiometer is a manual analog voltage source, while a sensor is an environmental analog voltage source. The same ADC and PWM logic can still be used if the sensor output remains within the safe Arduino input voltage range.

6.18 Answer Key

This section provides the correct answers or expected answer patterns for the questions in Chapter 6. The answer key is essential because it ensures consistent evaluation. Without an answer key, assessment may become subjective or incomplete. The answer key should include exact answers for factual questions, calculation steps for numerical questions, and acceptable reasoning points for explanation questions.

For concept questions, the answer key should confirm the following.

The potentiometer produces a variable voltage at its wiper. The Arduino reads this voltage through A0. The Arduino converts the voltage into a 10 bit ADC value. The ADC value is mapped to an 8 bit PWM value. The PWM output on D9 controls the LED brightness through a 220 Ω series resistor.

For calculation questions, the answer key should include the following relationships.

$$V_{A0} = 5V \times p$$

$$ADC = \frac{V_{A0}}{5V} \times 1023$$

$$PWM = \frac{ADC}{1023} \times 255$$

$$I = \frac{5V - 2V}{220\Omega}$$

$$I \approx 13.6mA$$

For troubleshooting questions, the answer key should state that if A0 is connected to an outer potentiometer pin, the input voltage becomes fixed. The LED brightness does not change because the Arduino does not receive a changing analog value. The correction is to connect A0 to the center wiper pin.

The answer key should not only provide final answers. It should also provide reasoning steps where calculation or diagnosis is required.

6.19 Marking Criteria

This section defines how answers should be evaluated. Marking criteria are required to make the assessment fair, transparent, and academically consistent. The criteria should reward technical accuracy, correct terminology, calculation method, explanation quality, diagram interpretation, troubleshooting logic, and extension reasoning.

A suitable marking system may use the following distribution.

Conceptual understanding: 20 marks.

Component identification and circuit knowledge: 15 marks.

Mathematical calculation accuracy: 20 marks.

Circuit scheme interpretation: 15 marks.

Simulation understanding: 10 marks.

Troubleshooting reasoning: 10 marks.

Extension and critical thinking: 10 marks.

The total is 100 marks.

The marking criteria should evaluate whether the learner can identify the function of each component, calculate ADC and PWM values, explain LED current limitation, interpret schematic diagrams, predict simulation results, and diagnose wiring errors. Full marks should be awarded only when the answer is technically correct, complete, and supported by clear reasoning.

For calculation based questions, marks should be awarded for formula selection, substitution of correct values, mathematical processing, correct units, and final answer. For troubleshooting questions, marks should be awarded for identifying the symptom, explaining the electrical cause, and proposing the correct repair. For extension questions, marks should be awarded for feasibility, safety awareness, and correct application of the original project principle.

6.20 Chapter Summary

This chapter presented the complete evaluation structure for the analog control project. The questions were organized to test understanding from basic concept level to advanced application level. The chapter began with conceptual questions, component identification, potentiometer operation, Arduino analog input, ADC calculation, PWM output, LED current limitation, circuit scheme interpretation, Proteus simulation, troubleshooting, short answer responses, calculation based questions, diagram based questions, practical testing, critical thinking, and possible system extensions.

The chapter confirms that the learner must understand both the theoretical and practical sides of the project. A successful learner should be able to explain the working principle, calculate expected values, identify correct wiring, interpret simulation behavior, diagnose faults, and propose suitable extensions. The evaluation structure therefore completes the academic report by converting the project from a simple circuit demonstration into a formally assessed embedded systems exercise.

The final conclusion of this chapter is that the analog control project is fully understood only when the learner can defend the complete chain of operation: mechanical rotation changes the potentiometer wiper voltage, the Arduino reads that voltage through A0, the ADC produces a numerical value, the program maps that value to PWM, D9 outputs the PWM signal, the resistor limits LED current, and the LED brightness changes according to duty cycle.

This page intentionally left blank.